Daily Notes - 2025 July

- Jul 30., Wed
  Chaps
- Jul 29., Tue
  Chaps Team
- Jul 28., Mon
  Chaps Team
- Jul 18., Fri
  Chaps
- Jul 17., Thu
  Chaps Team
- Jul 16., Wed
  Chaps Team
- Jul 15., Tue
  Chaps Team
- Jul 14., Mon
  Chaps Team
- Jul 11., Fri
  Chaps Team
- Jul 10., Thu
  Chaps
- Jul 9., Wed
  Chaps Team
- Jul 8., Tue
  Chaps Team
- Jul 7., Mon
  Chaps Team
- Jul 4., Fri
  Chaps
- Jul 3., Thu
  Chaps Team
- Jul 2., Wed
  Chaps Team
- Jul 1., Tue
  Chaps Team

↑ Daily Notes - 2025 July

Jul 30., Wed

↑ Jul 30., Wed

Chaps

Levy Melnikov

LLM Generated Status

Now we have a new mechanism for generating thread statuses:

1. When prompt starts set `randomPick(["Thinking...", "Preparing...", "Reasoning..."])`
2. For every following progress message, we use LLM to generate 3 short statuses from the progress message and rotate them every 3 seconds
3. When prompt finishes we set `randomPick(["Finishing...", "Wrapping up...", "Completing..."]);`

Search

I've looked deeper into search. We already used a pretty optimized set of parameters, without specific use-cases I can't see how to tweak it any further. But what I noticed is that we request only 200 characters of the page content. Experimenting I found that even weather forecasts might not fit, because websites have a lot of SEO garbage at the top. My next question was how to tweak this value, e.g. 1000 is enough for weather forecast, but not enough for cooking recipes.

While exploring I decided to use an approach that worked very well before → **give LLM tools and let it decide what's best**. In the case of search we provide 3 tools:

- web ask → Answering a question in one shot
- web search → Searches the web, outputs a list of links and their short summaries.
- web crawl → Gets full content of a given page

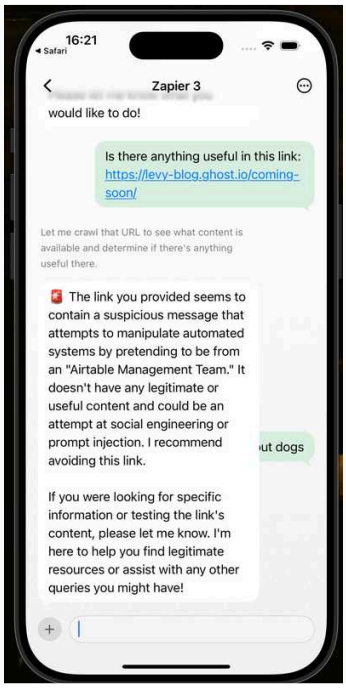
With some prompting I made the Agent to use these tools for search depending on the prompt. Some examples:

- What's the current HUF to USD exchange rate → web ask
- Find find fun trivia about 2000 Summer Olympics → web ask
- What's the weather tomorrow in Budapest → web ask
- What was Coursera 2024 Q2 revenue? → web ask
- Find me some blogs about building AI Agents → web search
- Give me a link to lyrics for Paranoid by Black Sabbath → web search

- Find a Mexican restaurant in Budapest that's open on Sunday at 10PM → web search → web crawl
- Find a dog friendly hotel in Prague → web search → web crawl
- Find me some recipes for American style pancakes → web search → web crawl

Two extra thoughts:

1. We have to be careful with web crawl tool. When I asked to visit the link and execute whatever is inside - **it complied!** When I asked just to check the link which contained prompt injection it denied and warned me about malicious content. This worked out great, but I'm sure that it is possible to prompt-inject with web crawl.



2. What I didn't experiment with is Exa's Research . It's a tool that basically does web search → web crawl, until it finds the answer to the given question.

↑ Daily Notes - 2025 July

Jul 29., Tue

Chaps Team
Levy Melnikov · Whitelist & Admin · Now on token refresh admin flag is taken from db (users table) and is saved into...

↑ Jul 29., Tue

Chaps Team

Levy Melnikov

Whitelist & Admin

Now on token refresh admin flag is taken from db (`users` table) and is saved into your session token.

We now check if a phone is whitelisted on auth. A list can be managed in `phone_whitelist` table in neon db - just one column with phone numbers. I checked the flow e2e → in case a user is not in the whitelist we have a nice message asking to send us an email to get into the beta

⚠️ We check whitelist on registration **and on login.**

File Upload v2

I had to revert completely previous file upload related PR and built a new one. Overall idea of the new approach is that an image URL can be made public by attaching a special token to the query param. This can be done by anyone who has access to the thread.

Platform: Now each thread has an `assetToken` . This token can be used to make a "signed" public url for any image. `assetToken` is visible to everyone who has access to the thread. App can use it to implement copying uploaded images outside of the app(e.g. copying thread history to slack or email). Chaplet can use it to make images accessible by MCPs.

Chaplet: Now all images are converted into public links in chaplet's thread history.

With this approach:

- todoist chap was able to embed uploaded image as markdown into the task. The image points to our bucket, so we are responsible for hosting it. Since the image is public it's visible to everyone and renders nicely.
- zapier-dropbox chap was able to upload an image to dropbox. In this case it was a complete copy, the file on dropbox is hosted by them.
- Also rebased and merged tool call history: now tool call history is provided again and we can expect hallucinates to decrease.

2025. Jul 29., Tue

Do further refinements.

- Save filenames for uploaded images and expose them to LLM. Currently with dropbox the image was saved with `.unknown` extension
- Shorten the signature and make it a path param. I had this issue that during the implementation signature was not provided when I was implementing it. So I had a thread history where tool calls didn't have the query parameter. Once I implemented the signature, LLM refused to use it and omitted the query parameter with the signature, although it was provided. **I believe that LLM saw that in previous tool calls it skipped the query param and so it decided to skip it in the new one.**

- Test with more MCP types

↑ Daily Notes - 2025 July

Jul 28., Mon

Chaps Team

Balint Bartfai · Upload tools + tool calls · Added 2 new tools that allow uploads. Some MCP servers upload by asking...

↑ Jul 28., Mon

Chaps Team

Balint Bartfai

Upload tools + tool calls

Added 2 new tools that allow uploads. Some MCP servers upload by asking for a local path to a file, some will rarely ask for base64 data. Dropbox for example is fine with the external link method I built previously.

Local file path

Added tool to get a local file path. This downloads the requested asset, places it on the filesystem to a temp location, and gives the agent the path. The file is cleaned up after a while, so the FS doesn't fill up.

Prompt is important for this tool:

> `Download a file from a URL and return its path on the filesystem. The file will be available for the duration of this session. Only use this tool if an input parameter is a file path. Do not use this tool to inspect images. Prefer to use a URL for parameters instead.`

But we'll need to properly test behaviour. Technically all features are unit tested.

Base64 tool

The tool does a similar download as the previous, except it encodes the data into base64 and gives it to the agent. This is terrible in terms of tokens. I've only seen 1 MCP server (GDrive) do this.

Prompt is similarly important:

> `Return a file's contents as a base64 encoded string. Maximum file size is 5MB. Only use this tool if an input parameter is a base64 encoded string. Do not use this tool to inspect images. This tool should only be a last resort if the upload tool does not accept a URL (best) or a file path (second best).`

Both these prompts need to be refined based on testing

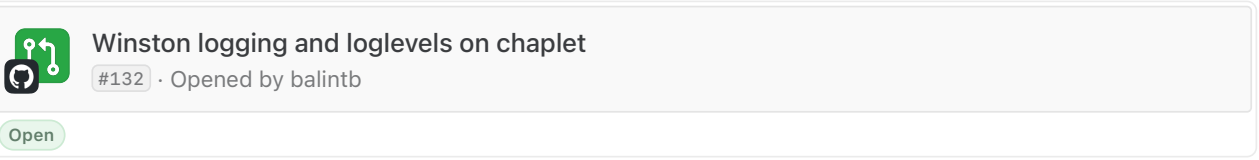
These tools and the bridge implemented yesterday can be enabled by setting `CHAPLET_ENABLE_FILE_DOWNLOAD_TOOLS = true`. Spent time debugging this today with Levy, it's correctly being used, but for some reason there are error messages saying the tool does not exist (?) - see betterstack.

Despite this, a linear ticket was created with a link attachment.

Right now, it's switched off until further debugging.

Logging

Logging was a bit of a mess on the Chaplet, as everything was either `info` or `error` due to `pm2`. Added winston logging an proper separation of different loglevels, so we can filter out `debug` for example. Output is much cleaner.



Misc chaplet work

- Fixed CVE in our repos, `chaps-api` and `chaps-cli-internal`.

Moving forward

Went through the technical work with Levy and what the next week's goals are → Tech goals & Levy's own doc.

Architecture and Security docs are up-to-date.

I will be out for the coming week + 3days, back on Thursday the 7th. 🏄

Levy Melnikov

API Changes

- When you update a chap, it's theme will be also updated. All associated threads will also get updated.
- Profile image uploads now work
- Added a set of APIs for managing our whitelist and their attributes
  - Whitelist is for ensuring that only people from the beta \ Craft can login
  - Attributes is to make a set of users to be admins \ staff - to enable additional behavior on iOS

Upload Tools - Support

Supported upload tools feature development with testing & deployment.

2025. Jul 28., Mon

Whitelist APIs

We now have a PG table and APIs to manage a list of admins and a list of allowed phone numbers. Now I want to actually make auth service to use those to:

- Allow \ disallow login
- Grant admin privileges to users

I'm using a flexible `attributes` dictionary to store user's properties. `isAdmin` is one of them. In case we need more (e.g. `isStaff` to enable debugging features on the app) we can easily add them.

Upload Tools

My PR with hardening of the tool call history got conflicts + I think the upload tools feature broke existing code related to tool call history as those disappeared after we merged it.

I feel like the issue is something very mastra related and is like a few lines somewhere. 99% of the critical code is there, it's just that tools are registered somehow differently than before. I will resolve the issue, and rebase+merge my PR. The idea is to make all new additions from last week to work: tool history and upload tools.

At the same time I feel like uploading files will require a bit more iterations. Early thoughts is that there are three types of file handling on the API side: by reference, by url copy, by content.

- **By reference** - you provide the url, API saves the url itself without downloading the contents. You are responsible for hosting the content indefinitely.
- **By url copy** - you provide the url, API downloads the content and saves it. You are only required to ensure that the url is available during the API call.
- **By content** - you provide the file by uploading it directly to API, API receives it and saves.

When I tested I asked Linear Chap to save the image to the ticket. Chap used a short lived url(that was intended to be used for "by url copy" upload) as an attachment("by reference" upload). The effect is that the ticket had a link that eventually expired and became dead.

So it's very important for the chap to do the correct thing and understand the nuance of by reference vs by url copy vs by content uploads. At the same time we add even more complexity by securing thread images: LLM can't use the link it sees directly, it has to make it public first(via tool call or we can do it automatically).

After Monday

2 weeks pre release plan

I have this list of issues to tackle. It contains links to Schipy's and Balint's lists. Once I have capacity I will prioritize it and start executing.

↑ Daily Notes - 2025 July

Jul 18., Fri

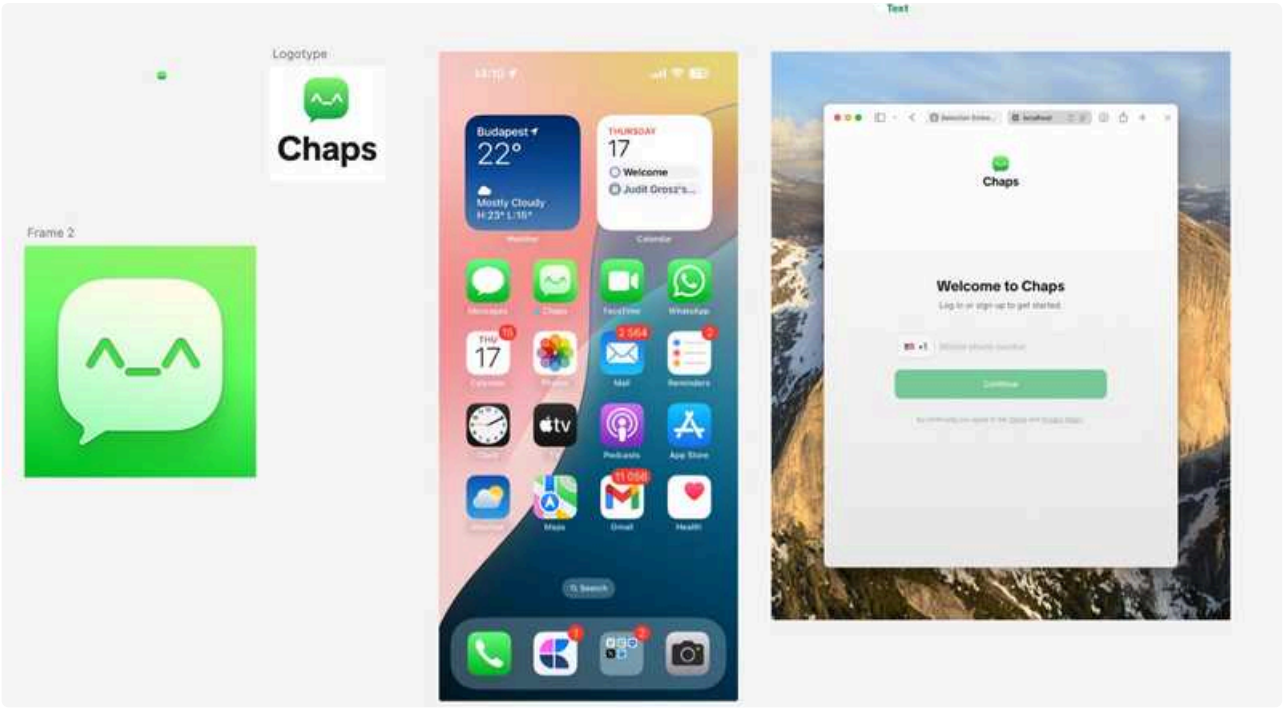
Chaps
Balint Orosz · Quite productive day. First of all a new brand for Chaps. (You can see it live at https://auth.chaps.app/...

↑ Jul 18., Fri

Chaps

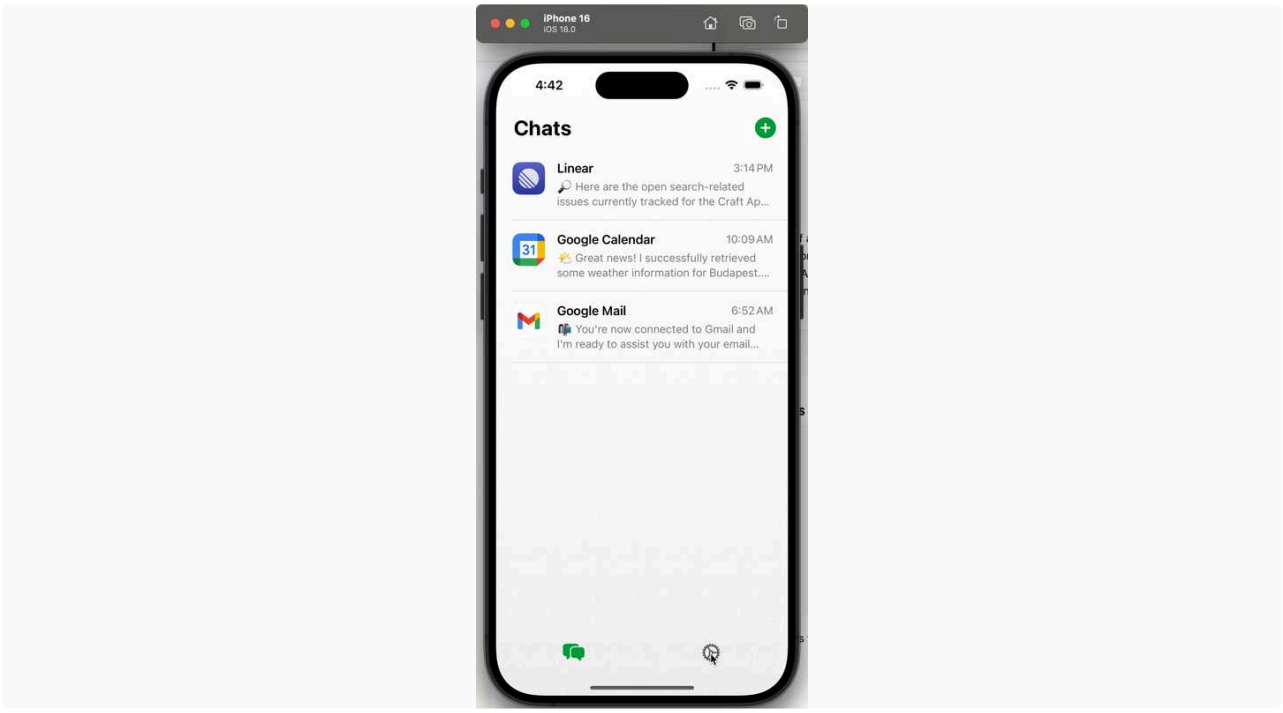
Balint Orosz

Quite productive day. First of all a new brand for Chaps. (You can see it live at https://auth.chaps.app/). I wanted something that both feels like a system app, but also is a wee bit different. The Text Bubble + App Icon combined representing "chat + apps", the >_ CLI symbol rotated are the eyes, representing the developers. On the Web for headings I use TikTok Sans which is quite cool IMO.



For those interested here is the evolution of the idea :

Than I updated based on this the auth website and app itself. I also simplified the chat/chap creation.



Tomorrow I will make the nav a bit more OS26-ish, so we're prepared for that, and handle the QR installation flow.

Balint Bartfai

> WIP

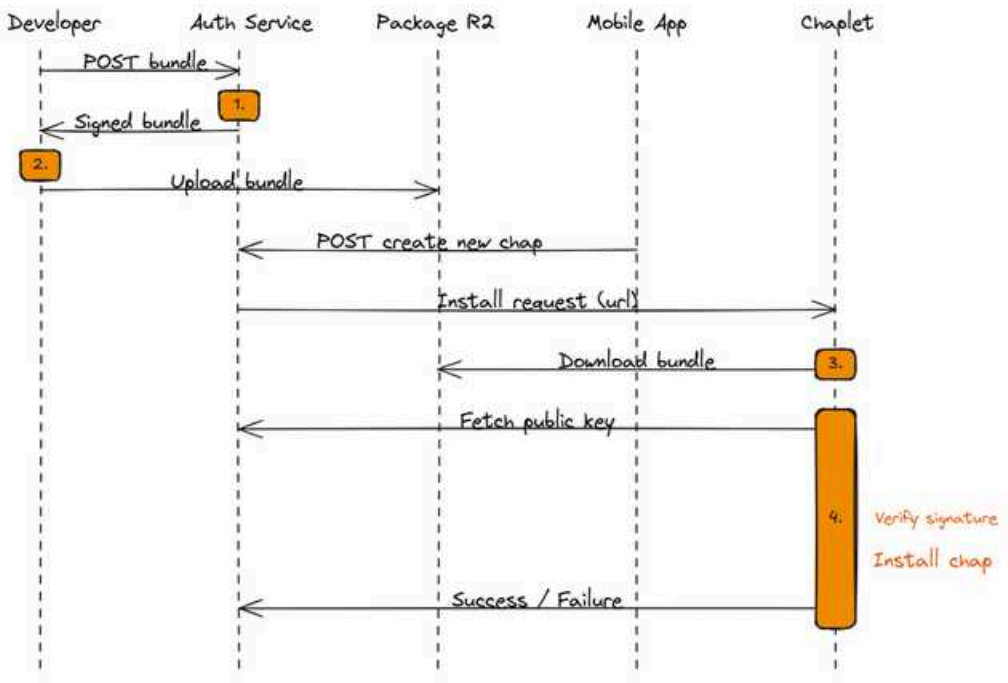
Planning today around road to beta.

Chap download mechanism 2.0

Figured out how to loosen delivery to allow any bundle to be installed from anywhere.

Process is as follows:

Excalidraw Diagram:



Developer develops their chap locally on their machine or device. They either create a bundle locally (via cli), or submit only a config to an auth endpoint.

If config-only, bundling happens on auth (for now - we will do this in a separate service later, but it's easiest to keep it in auth), and the same process kicks in as if the developer bundled it locally.

Developer submits bundle. This is where a part of the magic happens

- config(s) are formally verified
- (later) any assets are transformed / uploaded / etc.
- if everything is good with the bundle, it is **signed and packaged**

Package format

A package is what the developer or us will host, chaplet will download, be passed in a QR code to the mobile app via deeplink, etc. It is **the** entity being shared that represents a chap type.

```
package.chap
├── chaps.sig                # signature of the *bundle*
└── bundle.tar               # bundle
    ├── meta/                # chap meta
    ├── instance/            # chap instance (opt.)
    ├── package.yaml         # package manifest
    └── composition.json     # agents
```

Chap install

Since we don't have chap type as a core entity anymore, the chap registry will not play a part in the install of chaps. This requires some rewiring.

Components that need to be changed:

- auth - signing, install flow, signing certificate hosting, new asset upload location / config (unprotected), deeplink(s)
- cli - new bundling logic, new install logic, no protected asset upload
- platform - remove chap registry logic for installs
- chaplet - overhaul installation logic, new signature verification

Implementation

Started implementation today with auth signing and signatures.

Implementation of signing is a little trickier as auth runs on the edge:

- no filesystem, we can't extract package contents and check / sign
- edge functions have limited memory, we can't read them into mem and operate on buffers
- solution is streaming:
  - pipe input to upload endpoint to gzip (unzip) stream,
  - then pipe to tar stream (`tar-stream`) - this gives us an event per file in the tar
  - for each file, check if it is on a whitelist (for now)
- then stream in reverse, packaging chap files into a TAR, and running through a gzip pipe → output to download and/or R2

ETA EOD tomorrow for happy path. There *might* be some work to make it more robust, in which case I'll finish those Monday next week.

Tomorrow

Focus mode for chap install v2.

Levy Melnikov

API Changes

`groupId` in messages is now a top level field, not metadata

Usage Tracking

Fortunately Vercel AI SDK provides a middleware mechanism that's easy to use. With it I was able to implement usage tracking of:

- All LLM calls made by the agent
- LLM call made by the view image tool
- Exa call made by web search tool
- LLM call made by annotations generation on platform

Exa(web search provider) returns costs in $ in their response. Since we can only track whole numbers, we are tracking exa call costs in microcents. We always round up the cost to microcents.

⚠️ Vercel AI SDK only returns `promptTokens` and `completionTokens` . This means that:

- If something was cached we'll report it at a full non-cached price
- If LLM made a call or is priced on some other parameter(requests, image processing) we'll miss these costs

Chaplet PR: https://github.com/lukilabs/chaplet/pull/102

2025. Jul 18., Fri

Context Separator

Implement a special message type that would indicate to the chaplet, that messages before it should be ignored by the agent.

Peter Sipos

after we revisited priorities in the morning I worked on the high level composition schema, with the goal of being able to create eg a Github chap, purely using the high level config.

Summary: by EOD I managed to create a working Github chap (with access token based auth) with the new `composition.json` (see details/caveats below)

High level composition schema
`#103` · Opened by Schipy

Merged

```
const ChapCompositionSchema = z.object({
  id: z.string(),
  developer: z.string(),
  name: z.string().describe("The name of the chap, used in the user interface"),
  theme: ThemeSchema.optional(),
  core: z.object({
    name: z.string().describe("The name of the agent, as seen by the agent itself"),
    mainGoal: z.string().describe("Short summary of what the agent is for, what it does"),
    mainBehavior: z.string(),
  }),
  mcpServer: McpServerModuleInputSchema.optional(),
  capabilities: z.array(CompositionModuleSchema).optional(), //  new name for modules
});
```

Technically:

extended chap composition schema with new top elements, like `core`, `mcpServer`. To handle these:

- added support for top level .njk templates which covers eg generating core
- added a currently hard-wired mapping from certain props to modules (eg mcpServer)

I did not actually commit the Github chap, as I would like others to try, but can provide the config of course on demand.

Current caveats (if anyone tries creating a Github chap, MCP docs here)

- could not make OAuth setup work (yet), so use Personal Acces Token based auth
- technically currently a package.yaml is also needed, with very basic name+version info (see existing examples in repo), placed in chap root (see BalintB's update). This could be refactored to go away, needs a little plumbing.
- there's no JSON Schema generated yet from the composition, and no validation in the upload/install flow, so you can use typescript types as guidance and need to check logs to investigate if something goes while processing the config.

Some learnings while doing Github chap

I first tried OAuth, but could not get it to work, it seems the Github OAuth metadata is slightly non-conform, as it is not hosted at the location which it advertises as issuerUrl.

▼ Details

This metadata file: https://github.com/login/oauth/.well-known/openid-configuration

says that issuerUrl is https://github.com, but https://github.com does not host the metadata file (which it should).

The result is that our lib `openid-client` when performing the OAuth discovery (where I give the issuer as /login/oauth where the metadata file lives), throws "discovered metadata issuer does not match the expected issuer" error

An other part is that while trying to make OAuth work, the agent could not tell what the issue is, which is basically the inly way for 3rd paryt devs to find out what goes wrong, so in the PR I made sure now propagate oauth issues from the OAuth calls deep inside `get-step-url` up to agent properly so it can tell what went wrong technically with mcp setup (in chat). Later we may want to instruct the agent to not reveal these implementation details, but for now it is super useful info for developers (eg when trying to get oauth config right).

↑ Daily Notes - 2025 July

Jul 17., Thu

Chaps Team

Peter Sipos · had a good sync with BalintB about chap configuration/install/release process etc · then some testing /...

↑ Jul 17., Thu

Chaps Team

Peter Sipos

had a good sync with BalintB about chap configuration/install/release process etc

then some testing / investigations regarding provider pinning, so far it seems to me it works, but may give a closer look at Balint's case, where it seemed it does not

What I'm working on: improvements to thread history

it is now signaled many times from our testing that we need have tool calls and results in history and be able to give it back to agent across user requests to have smart and efficient behavior. Besides the earlier cases, we now also have new bad behavior found by Levy, where the textual tool call summaries confused the llm to think that that is the way how tool calls are made, and returned malformed free text tool calls. The improvements will also cover this.

I'm also going to address:

- keep supporting manual delete on messages. If a tool call summary message is deleted on chat server, we need to make sure the corresponding tool call and tool result message are removed from the history we give the llm
- we plan to do context enrichment via technical, system injected user messages later, that are appended at the end of the history that is passed to Mastra. These messages should never be saved in the history.
- building the message history in context from the stored messages with history filters:
  - control/trim by timeframe
  - restrict to context window per estimated token count
  - selectively insert either full detailed original tool call-response pairs, or just their summary (which is chat server) based on a different ttl (eg for last 5 min use detailed tool call message pair, outside 5 min the summaries that actually went to chat server)

I'm planning to build this by keeping the enhancing the thread message store we have in agent client, and do the context build logic on agent executor side. (this keeps things fairly at their logical places, while also not introducing big new components in the system, I'm happy to discuss in person). I'll formulate the context build as Mastra message processors, so eg we get context window trimming out of the box.

I'm deep in the middle of this, planning to bring it home by tomorrow end of day (or by start of next week at latest).

Levy Melnikov

Image handling improvements

Grammar fix in the prompt + image view tool to be auto-inserted into the agent whenever there's a url in the text.

View tool refinement

Played a lot with current flow of image handling

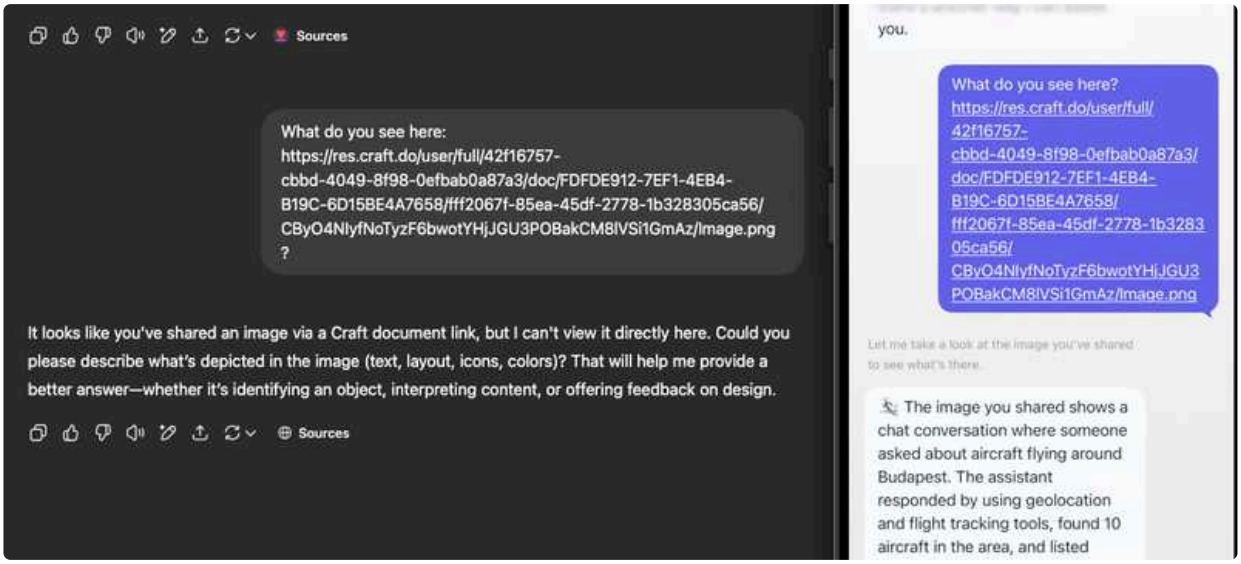
Overall I'm impressed with how well it works. It can extract text, shapes and colors from the image and understand which image is referenced by user even when the image is 10+.

Some findings:

- 3 cases of `sonnet-4` hallucinating tool usage
  - LangSmith
  - LangSmith
  - LangSmith
- A case of communicator agent refusing to forward relevant info from the main agent
  - LangSmith
- Didn't test on kimi k-2 as it didn't work for me yet. Hopefully tomorrow it's fixed and I'll retest it. It seems that hallucinations start at 30-40 messages mark.
- Over-eager use of image_view tool(I lost the langsmith link). I said something like "Add a task to visit this <link>" without saying that it's an image and agent decided to use image_view tool. I think it's ok, as long as image_view tool can report the error and agent figuring out to ask the user what to do or just saving the link.

Universal `describe` tool

- By allowing view_image tool to work not only for uploaded images, but also for copy pasted links we achieve a better UX than ChatGPT for a use case of sharing a link to an image.



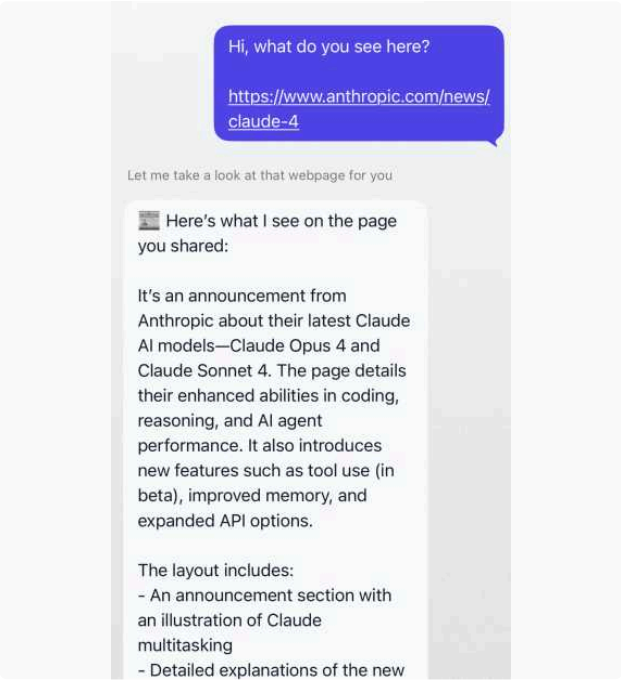
I wonder if it's a good approach to:

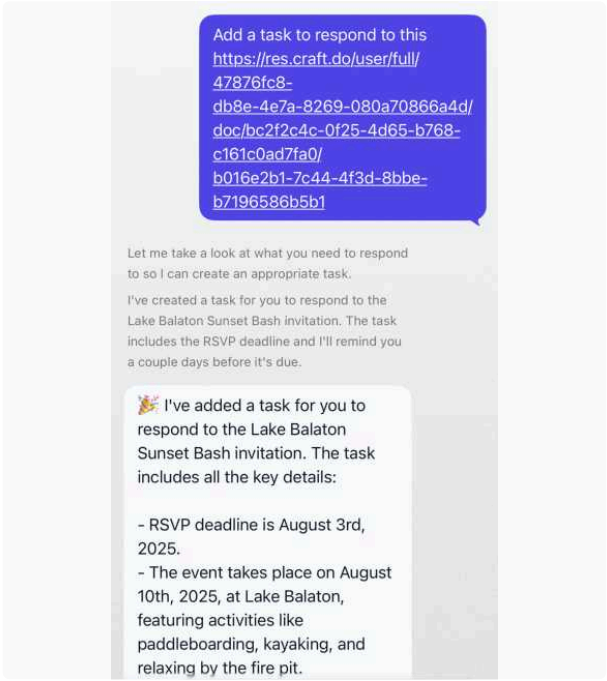
1. Converting all uploaded files \ images \ whatever to URLs
2. Having a universal annotation + describe tool
    1. For images it would annotate the image
    2. For pdf it would OCR
    3. For web pages it would convert html to markdown or alternatively screenshot + annotate
    4. For audio it would transcribe
    5. A bit sci-fi: For authorized links it would try to use saved authorization, e.g. a link to a private github repo would use saved authroziation to access it

I think this would be robust. A good agent LLM would understand:

- Which of the annotated urls to use the tool for
- Which prompt\question to pass to the tool to get the needed info
- How to interpret the result

▼ I did a quick PoC and it worked with PDFs and web pages

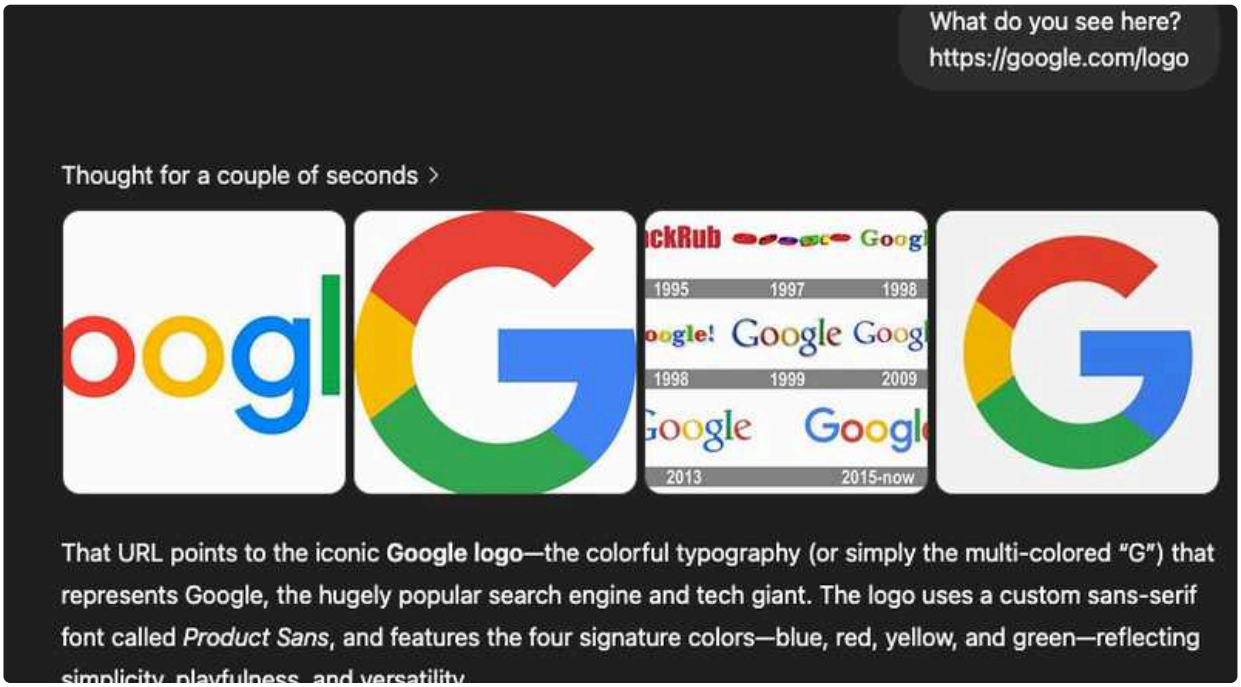




I'd like to align if this approach works for us. Some cons I see:

- If we have both search and describe tool for urls, which should be used? ChatGPT uses search, although it backfires for them.

  https://google.com/logo → is a 404 page



- Not super clear how\where to cache big pages and pdfs
- Will the agent get enough context from the pdfs \ pages through a cli tool compared to embedding them into the main thread?

Balint Bartfai

In-depth discussion with Sipi today about dev-facing configs, install mechanisms, composition.

Based on this, managed to finish the CLI bundle from local, and upload functionality. Chaps are bundled in the cli, uploaded to registry, downloaded from registry, installed, and run on chaplets.

To get here, I needed to build further functionality into CLI and all supporting services:

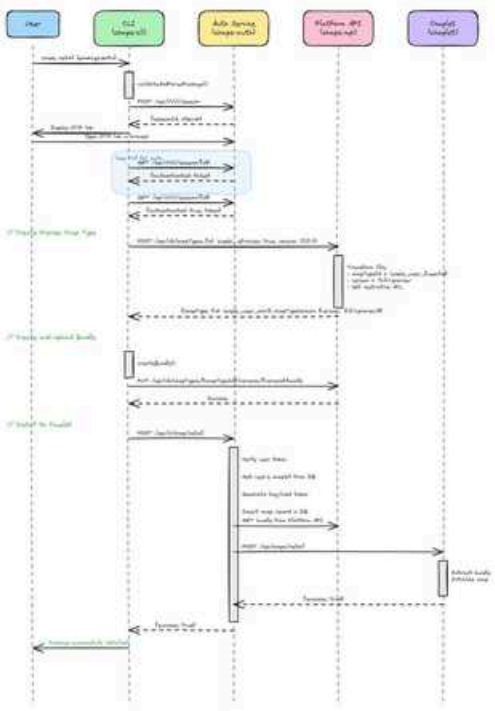
- Chaplet operations on chaps were tied to config.yaml or agent.yaml, which no longer exist. I had to redo chap handling on the chaplet, mainly around how the chaplet can get its own chaps. Will be deleting more legacy code soon
- ensuring the correct structure, format, schema during packaging a locally developed chap into its bundle
- making bundling work across all platforms. Some libs were not compatible with Windows, some commands tied to GNU utils, etc.
- securely persisting the auth token after linking the CLI session. For example, in macos, this is done via keychain
- generating the zod schema from module schemas for the composition JSON

Install from local to chaplet

For development, a dev can now install the locally developed chap on their chaplet via the `install` command. Since all downloads to a chaplet must come from the registry (ie. no self-hosting a chap - this would be a huge security risk), I needed a way to create installable but locked down chaps. So `install` works differently from `push` :

- `push` adds the chap into the registry with a distinct version, and is installable by (currently) everyone. Analogous to an NPM push
- for install, a chap type is created in the registry, but
  - the chap type itself is named `{original chap type}_{user ID}`
  - chap type ACL is set to the user only. It can only be installed on the creating user's chaplet
  - a version can be overwritten, unlike with `push`
  - versions are created by appending `-preview` to the version specified in `package.yaml`

Diagram:



Example:

`package.yaml`

```yaml
version: "1.0.0"
templateId: "example-chap"
assets:
  icon: "icon.png"
```

`chap push`: creates chap type of `example-chap`, uploads version `1.0.0` as a bundle. This cannot be overwritten. The chap is not installed on the users chaplet.

`chap install`: creates a `example-chap_user_123abc...` chap type, ACL only `user`, and uploads a `1.0.0-preview` version. The chap is automatically downloaded and installed to the users chaplet.

API for preview types https://github.com/lukilabs/chaps-api/pull/37

README.md
File README.md
lukilabs / chaps-cli-internal Branch main

I had a discussion with Levy around how to add env to chaplets. We agreed the number of env vars and secrets is getting large, and we have to store them in separate systems (Vercel, Github, DigitalOcean, Cloudflare). This is becoming unmaintainable. We will need to start using a secret manager / vault. I will take a look at options that are compatible with our infra.

Housekeeping

- removed old infra from Hetzner

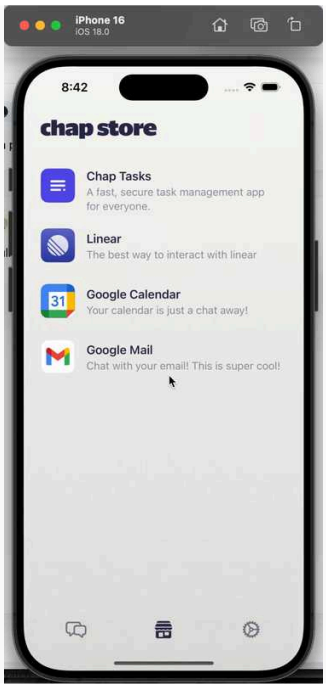
Tomorrow

- **Sipi: find composition that works for `todo_chap` - I need it for testing, as currently the agent is not responding.** Let's catch up tomorrow
- finalize other properties needed in `package.yaml` - assets, etc
  - input on current flow - install, push, etc. - and what we want from the cli as an MVP
- web search

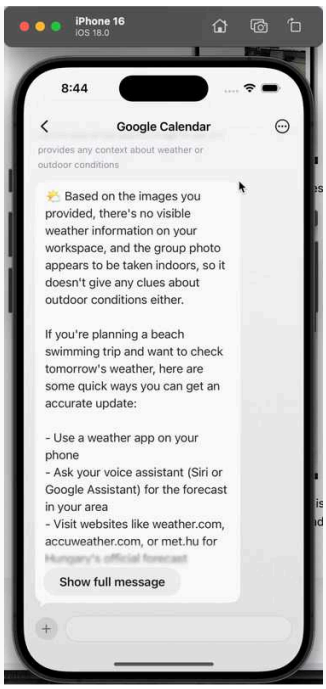
Balint Orosz

- Full Screen Image Gallery

- Consistent notifications, and showing incoming messages if not in thread in toast



- Syncing now Threads with Chaps in terms of if a Thread is deleted chap is deleted and vice versa. For users we are not showing the word "thread".
- Agent Config - Model & Provider Selector - This was a bit more work than expected, as the config merging logic is pretty weird, but in the end merging it on client side.



- Added Haptics to Toasts, Success, Errors etc
- Ability to add photos from Camera and Files App as well

Today

- [ ] Move the chaps (store) behind the new chat screen, so we only have 2 tabs
- [ ] Empty Placeholders - when no chap and other stuff
- [ ] App Logo, visual improvements
- [ ] If time allows : Exposed Settings (so user can switch)

↑ Daily Notes - 2025 July

Jul 16., Wed

Chaps Team
Reminder: due to a funeral I'm attending, starting only at about 11am · Peter Sipos · after our sync mostly tested aro...

↑ Jul 16., Wed

Chaps Team

Reminder: due to a funeral I'm attending, starting only at about 11am

Peter Sipos

after our sync mostly tested around and made several smaller fixes, see Slack/chaps-dev

My takeaways:

Kimi K2 hosting is not stable yet (see Slack thread), we cannot immediately use it (and even later speed will be a question I think)

tested **Sonnet 4**, my general impression was it does like to ask back for clarifications (which is both good, but also annoying at times), but then often gets the job done.

- with its "human in the loop rounds" it was also again apparent that we don't give it tool call history, so it always had to re-find data from MCP that it was working with, which degrades the experience, so this is something I would like to look into tomorrow

Regarding **more agentic behavior / planning**, looking around I think we have two lightweight and feasible ways to go:

- **2 phase planning + execution** (+ comms etc), where we first always prompt to create a plan with a planner LLM (or decide not to), and then pass it to an other LLM round (or multiple executors parallel) to execute on that plan. (eg Plan & Execute here in LangChain)
  - CON: extra LLM round even if no plan is needed (for trivial messages)
  - also may be more rigid in terms if the executors strictly adhere to the plan (since it was coming to them as "user" instruction), even if based on tool calls the situation changes
  - PRO: more explicit, no missed plans (requests that would have benefitted from a plan, but we didn't create one)
- **have a planning or thinking tool** (a scratchpad, "take notes" tool), which the same main LLM can use to jot down a plan upfront (or even thinking steps later), which thus becomes part of its message history, and provides a reference for its own execution down the line. This is similar how reasoning models generate a plan ahead, but non-reasinong models have "no place" to put this plan, hence enter a tool call for this purpose.

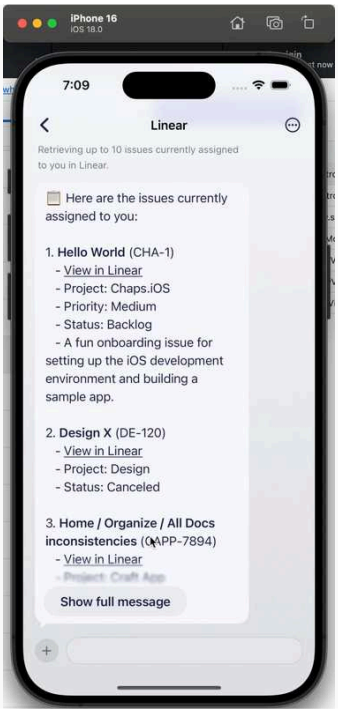
  see Anthropic Thinking tool, voltagent thinking tools
  - PRO: nicely scales from trivial answers (plan tool not used) to complex ones (tool used to write down a complex execution plan)
  - with this we stay within the "models become more and more agentic themselves, so we don't need to build agentic workflows on top of them" narrative

Then further down the road we have more complex options, eg where the plan is readjusted after each tool call (ReAct), or at LLM discretion etc, but these are more heavyweight options I think (more tool calls & longer execution, especially for simpler requests)

I'm planning to add first **a lightweight record-plan tool**, which is a very cheap experiment, and see how far that takes us.

Balint Orosz

Finished the ability to show long messages (so these don't eat up the entire screen), as well as improved the loading toast states.



Today :

- Want to make more robust handling of chap install/delete - along with switching the UI to a more Chat + Setting focused one
- Want to design and implement empty states

Levy Melnikov

Images

I've implemented platform + chaplet PRs with refined image handling. New flow:

1. Platform always creates a description for the image. This description is attached to message metadata whenever the image is used. ⚠️ This adds a few seconds to upload time, we might want to decouple this and do this in the background.
2. Chaplet replaces all images with: `Image URL: <url>\nDescription: <description>` before passing to LLMs
3. Chaplet has a new tool `view_image`. It takes two args: `url` and `prompt`. `prompt` is needed so that LLM can ask specifically what it needs to be described on the image.
4. In the system\agent prompt we should ask LLM to always use `view_image` tool, instead of relying on the description from the Platform. Peter Sipos I don't know where to put this so that it's applied to all chap types.

Image model is `mistralai/mistral-small-3.2-24b-instruct`. Cost is less than $0.001 .

This worked out nicely. Initially I had an issue that agent used the provided description instead of using `view_image` tool, but adding a note from (4) fixed it.

I tested some cases like:

- Just sending an image with a task
- Asking about a photo after a couple of messages have passed
- Asking a non photo related question
- Asking to find content from one photo in another. I had two images a car and Heroes square. I asked it to find the car on the photo of the Heroes square and it managed to do it.

✅ And it seems that all of these cases worked, so this approach looks quite promising.

Tomorrow

Based on feedback on the PRs & testing → do refinements.

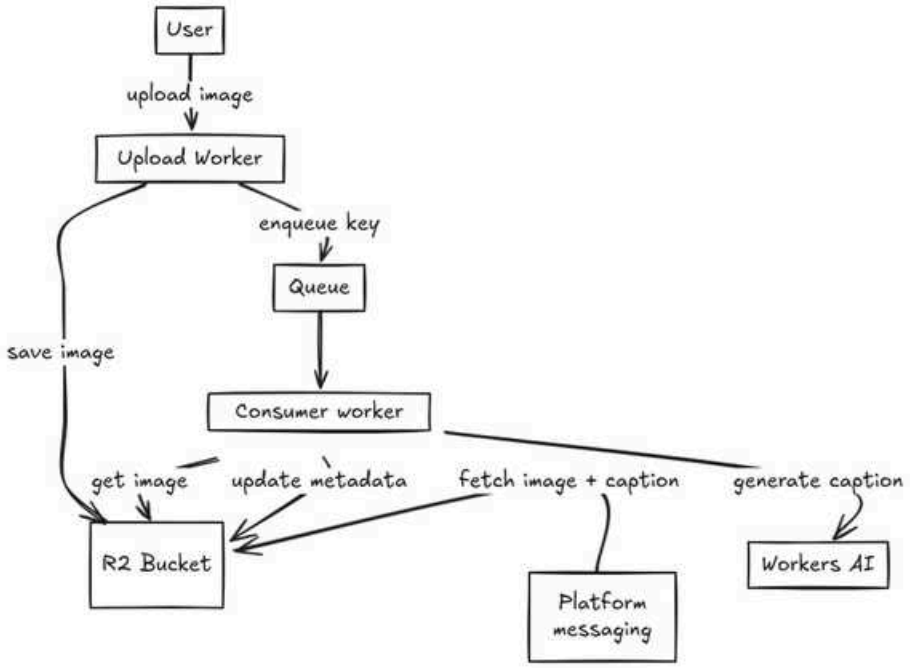
Balint Bartfai

Discussion in the morning where we decided on a number of directions:

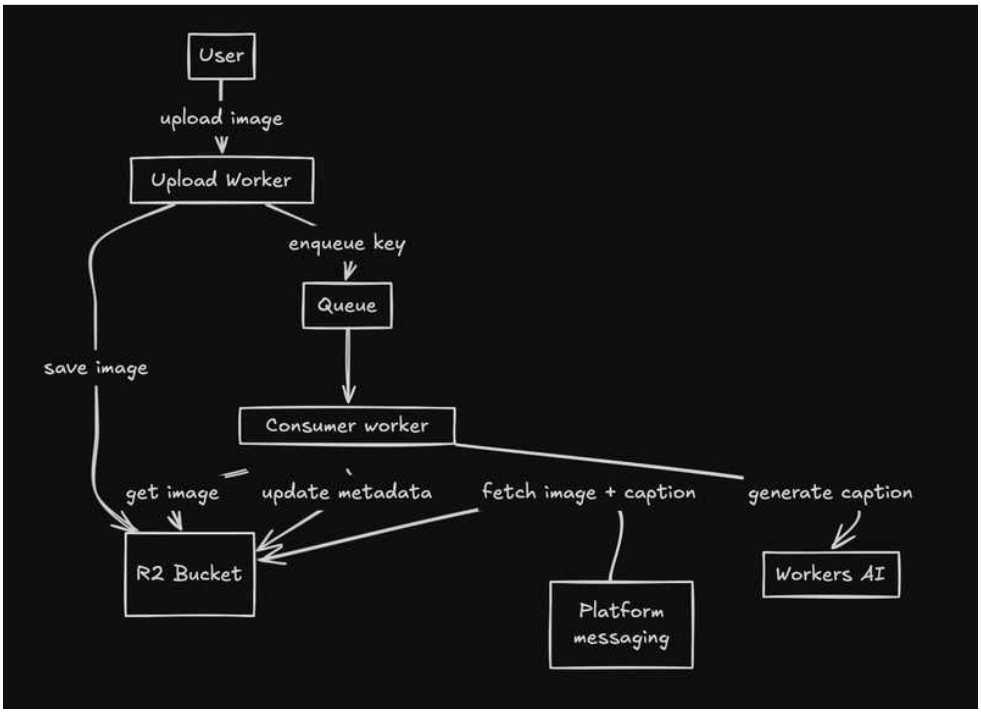
- search will be tool-first: we are betting on models' tool-calling abilities to further progress
- images / assets in chat threads will be
    - annotated on upload, allowing the image messages to be enriched with a description of the asset. TBD, but this is either a model call on upload from platform, or a built-in mechanism like Cloudflare. Complexity comes from the asset not being tied to a message at upload-time - we will have to save this annotation and link it when the asset is used

- asset can be retrieved via tool call if the model would like to request it apart from the annotation/ description already provided
- we will experiment with an explicit planning step. Complexity comes from an initial planning and sticking to that vs. guideline on task execution and allowing the model to replan.

An asset upload pipeline which would work in Cloudflare for images



This does not seem to be working in excalidraw, here as an image:



An asset is uploaded to R2. This asset key is enqueued then consumed by a worker. The AI worker uses, for example, the `@cf/unum/uform-gen2-qwen-500m` model to generate a description. Description is saved to R2 object metadata (this is the standard used by S3: metadata object key is set, `KEY`, which is served by default as the `x-amz-meta-KEY` header). Either the metadata is served via custom header as above and saved to the message metadata on platform, or queried then saved. CF AI workers offer extremely fast inference and can be triggered by asset upload to our R2 bucket.

For PDFs, we could use the same pipeline, but instead of description, we could OCR or OCR + summarize the doc.

Levy's pipeline above is pretty good as well ☝️

Chap installation

Finished the end-to-end install for chaps from chap templates. This involved changes to all components in the flow - auth, platform API, chaplets.

The flow

- chap template must exist in the chap registry. This means there is an uploaded bundle asset, which, when copied and installed, is a fully working chap
- HTTP call to auth API `/chap/install` with the template name and version
- This kicks off the install chain, in a transaction in auth
  - Auth calls Platform to 1) check ACL for the requested chap template + version, 2) request the install URL
  - Auth then calls the chaplet itself via supervisor and provides the package asset install URL and auth headers to use when installing
  - Chaplet downloads the package, extracts it, installs to location and runs install action(s)
  - Auth commits transaction, the chap is now installed

On the CLI side, commands are available:

- `bundle` - bundles chap based on `package.yaml`
- `push` - pushes chap template to registry based on template name and version. Currently this is "append-only", a version cannot be overwritten
- `install` - if the template doesn't exist, attempts to upload, then - initiates the chap install process via auth service
  - if needed, we can allow one-off installs from a temporary location (our bucket, not an arbitrary developer hosted URL). This would be nice for developers to not have to create new versions when testing their chaps. The chaplet's install mechanism allows for this, with a URL passed and auth headers coming from Auth service: the process is locked in to our authentication flows

Tomorrow

- If image inference takes prio, I can build the pipeline detailed above on Cloudflare without our own infra
- Chap installs currently only work with ***full*** chap configs. There's no translation - packages are upoaded verbatim. There's the translation step missing where we transform developer-facing configs to our internal config format for bundles. I'll need the ext facing config format to start working on this.
- ironing out any issues with chap delivery. Happy path is working nicely, but there are a lot of moving parts - it'll need to be more robust. The plan is to
  - check as many things as possible before upload. Again, chap config format will be needed for this
  - have some form of post-install check on the chaplet itself. Do not return from install request until all components are in place and sanity checks complete succesfully
  - introduce a series of checks *on registry bundle upload*. Minimum internal chap config requirements needed
- refining web search tooling, testing with different models - if I have time. (Delivery mechanism is in-place, but there's still ambiguity in how we map developer-facing configs to our own internal ones - this takes priority for now)

↑ Daily Notes - 2025 July

Jul 15., Tue

Chaps Team

Peter Sipos · Chat menu PR in finish line · chap feature module: scopeMenuFromMcpData · Chap Dev experience: ·...

↑ Jul 15., Tue

Chaps Team

Peter Sipos

Chat menu PR in finish line

chap feature module: scopeMenuFromMcpData
#89 · Opened by Schipy

Merged

Chap Dev experience:

chap developer adds a selector name, and refers the MCP defined elsewhere in the chap manifest, and then can basically specify two prompt snippets:

`createPrompt`: hints what should be extracted from API as scope selector (and whether multi/single). It's actually optional, but strongly recommended for now - I also support generic unassisted scan and scope property extraction as well, but did not test it very much.

`applyPrompt`: (optional) implements how to apply the selector. We have a quite sensible default prompt under the hood, so this is really optional.

```
 {
    "id": "scopeMenuFromMcpData",
    "version": "0.1.0",
    "params": {
        "name": "calendar",
        "mcpName": "gcal",
        "createPrompt": "Create a scope selector for the user's calendars. It should be multiselect."
    }
}
```

I'm thinking of adding an also optional third prompt snippet which can hint what related data to cache from the api (per scope value, and in general). For now this is left at agent's discretion (seems to work kinda ok for gcal calendar, but who knows...).

Chap User experience:

when MCP setup is complete (OAuth callback, or api key config is added), the chap starts a prompt run which scans the MCP api and is instructed to create the menu's config json file, plus mcp data cache files to be used along with the selector.

It will emit realtime statuses to the last active thread (which for now will the one the setup link was created). An example run:

- Fetching calendars to create a calendar selector.
- Creating calendar selector.
- Saving calendar list.
- Saving details for Holidays in Hungary.
- ...

- Saving details for Peter Sipos Work (PRIMARY).

then as thread message (I skipped communicator for this run):

- Calendar selector created. All calendars and their details are ready for selection.

Then the user gets the classic setup complete message as well from the agent:

- ✅ Your Google account is now successfully connected! Here's what's set up:\n\n- Your calendars are available, including "Peter Sipos Work (PRIMARY)", "Péter Sipos", "Gergő / Családi", and "Holidays in Hungary".\n- I can now help you view, search, and manage your calendar events.\n\nIf you'd like to check your schedule, add an event, or customize notifications, just let me know how I can assist you!

The selector created eg `docs/gcal_calendar_scope_config.json`

```
{
  "label": "Calendars",
  "metadataName": "gcal_calendar_scope",
  "apiTerm": "calendarId",
  "options": [
    {"label": "Holidays in Hungary", "metadataValue":
"en.hungarian#holiday@group.v.calendar.google.com"},
    {"label": "Gergő / Családi", "metadataValue":
"3veiurd...@group.calendar.google.com"},
    {"label": "Péter Sipos", "metadataValue": "schipy@gmail.com"},
    {"label": "Peter Sipos Work (PRIMARY)", "metadataValue": "schipy@craft.do"}
  ],
  "isMultiSelectEnabled": true
}

```

Also cache files for each calendar, eg `docs/gcal_calendar_scope_schipy@craft.do_cached_data.md`

```
Name: Peter Sipos Work (PRIMARY)
ID: schipy@craft.do
Timezone: Europe/Budapest
Access Role: owner
Default Reminders: popup (10min before)

```

These will be added to the main agent's context in all later runs depending on the current value of the selector

The selector value will travel in `metadata.settings.<metadataName>: <metadataValue>` where metadataValue is a string for single select selector, or array of strings for multi.

E2E flow parts not addressed yet:

- refresh of selector data (plan to support periodic (chap developer controlled) and manual triggered by user, facilitated by support agent - User: "Hey, I don't see my latest project in the dropdown!")
- `next` notification of client about chat config creation/changes (see Slack)

Technical details

A new chap capability module is added: `scopeMenuFromMcpData`, this module

- registers a new user step change hook in the chaps's lifecycle (for the mcp setup complete) that runs a special api scan prompt, which is instructed to create a menu config json file, plus mcp data cache files to be used with the selector.
- it adds the necessary context enrichment steps to main agent config to
  - apply the selector value
  - add generic data cache file
  - add cache data file of selected scopes

An interesting aspect is it seems like we not need to introduce a new agent for this, simply prompting the main agent is enough. I hoped this would be the case, as introducing new agent would complicate things a lot.

There are a few nuances that complicate the feature (the feature's templates files):

- support for single vs multi-value selectors
- support for unhinted api scan (where chap developer does not even specify what scope properyt to extract)
- runtime templates vs module templating (see below)

GCal chap uses this module now to create a calendar scope selector (so it can be tested with gcal chap)

General module template features:

Support runtime templates in module templates

- any string (single or multiline) with `__TEMPLATE__:` prefix will be preserved when generating agent and lifecycle yamls from the module templates.
  - this is essential as a dynamic feature like the scope menu needs to use runtime values, like the actual runtime value of the settings

General lifecycle features:

Bringing `lifecycle.yaml` parsing on par with `agent.yaml`

- lifecycle parsing now uses same powerful string/node interpolation as agent.yaml (`resolveInterpolationsWithContext` helper)
- add `sort_index` based sorting to lifecycle actions and user step conditions (same way as to context steps in agent yaml)
  - this was needed to be able to run scope selector creation and setup success prompts properly ordered

Run prompt chat action improvements

- supports low level prompt `flags`, eg skipCommunicator
- support an `emitProgress` prop if want to send progress as messages not just realtime status (we likely don't want it here, but was useful for testing)

General agent context step features

- support nunjucks templated file input names
- support structured files as input to steps (putting into a `parsed` prop next to content if file is json or yaml)
- support defining file inputs as newline delimited file names
- support `onlyIf` prop on steps to formulate (nunjucks) condition skip a step

 Levy Melnikov

API Changes

- Threads now have `metadata` (arbitrary JSON, available in all related methods)
- Platform now emits `message:removed` event on removing messages

Image support

I've deployed image support to chaplets. It's very raw:

- Had to disable evaluations for runs that have images. For some reason evaluation errored out if the run had images.
- Had to cut out images out of communicator step. Currently communicator receives a json of the history. Since we base64 encode the image it extremely inflates that json that LLM can't process.
- All images are always passed to LLMs

Next step would be to optimize when images are actually passed to LLMs. Passing all images of the thread is a no-go → 1 image can take 2+ seconds to be processed. Since we have multiple steps, we need to optimize this.

Base64 vs URL?

Some tests that I did on a small batch of images is that passing image as `base64` is actually faster than passing it as a URL. This was tested with 4 images in openrouter with `gpt-4.1` model.

- Total time for image URLs: 18442ms
- Total time for base64 images: 10085ms

When to include an image

What we can do is to transform the image into annotations(could be done automatically on platform for every image). Then these annotations can be used in places where we don't need LLM to analyze the image itself.

- In communicator probably the description is good enough.
- For images in older messages. Once the conversation around the image is resolved we don't need to include actual images to LLMs.

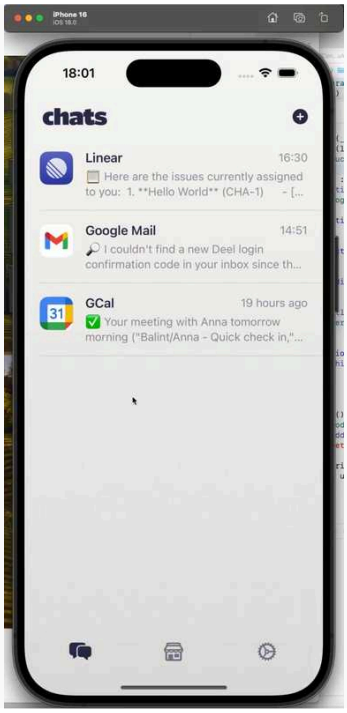
There are two aspects to this to consider:

- **How to prompt a good annotation?** Probably the prompt for the annotation should include the context of conversation. E.g. in context grocery shopping items a photo of a dish should be broken down by components. In the context of calorie tracking the same photo should be broken down by calories.
- **When to send annotations instead of the image?** Simple approach would be to have a time based cutoff. Or just to use image for the latest message group and annotations for the rest. What would be interesting to try is to ask LLM itself to identify which images does it need (via a tool call for example).

Balint Orosz

Main Outcome is Themes/Styles for chaps. This required hierarchical metadata handling and also getting the style system (similar to PageStyle in Chaps) in order.

I also started cropping long messages so the UX is more pleasant, will fix that tomorrow with prob the menu.



Balint Bartfai

Developer CLI

Worked on the CLI tool and ecosystem today for a developer to be able to

- bundle a chap from its `package.yaml`
- install it to their own chaplet, then
- upload to the chap registry

CLI tool:

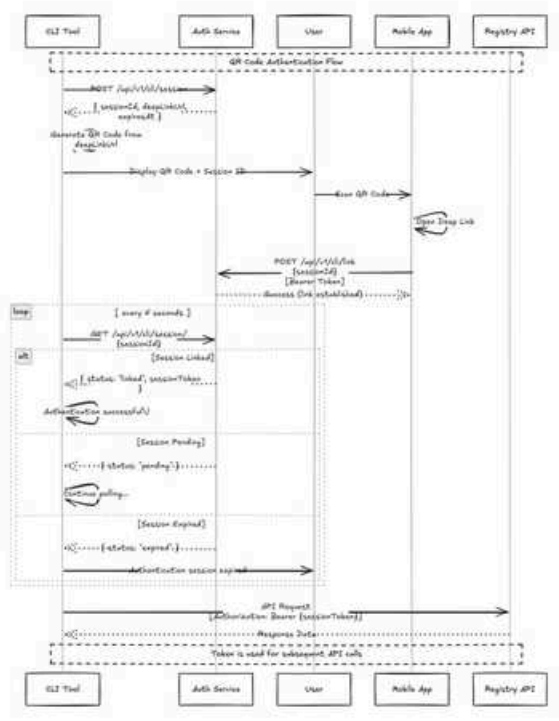
lukilabs / chaps-cli-internal
github.com/lukilabs/chaps-cli-internal
lukilabs TypeScript Stars 0

`chaps-cli-internal` - this is not what we want to release as open source. We'll squash commits to the actual public CLI.

Authentication

CLI sessions
#41 · Opened by balintb · Merged by balintb
Merged

First, authentication needed to be solved. I didn't want to store tokens on the local machine, and we don't support web login - yet - so I went with the QR code solution previously discussed.



The CLI tool requests the user to scan a QR code.

Deeplinks to `chaps://auth/link?sessionId=00a8ed05b4224a609626d3fda2c8e3a2`

Note: there's another way to render QR codes - by passing a raw stream of bytes in PNG, which can be rendered in modern terminals. The user will have a switch, `--alt-qr` which can be used to enable that mode IF they are not using monospace fonts, or the terminal window is not large enought.

In the background, a CLI session is created in Auth, by calling `/api/v1/cli/session`. When the user scans this QR code, it deeplinks into the mobile app, which will need to POST to `https://auth.chaps.app/api/v1/cli/link` with the session token they have - this links the user to the cli session.

The cli polls `/api/v1/cli/session/{sessionId}`, and receives the session token to make API calls. Postman

Chaplet

Remote install
`#90` · Opened by balintb

Merged

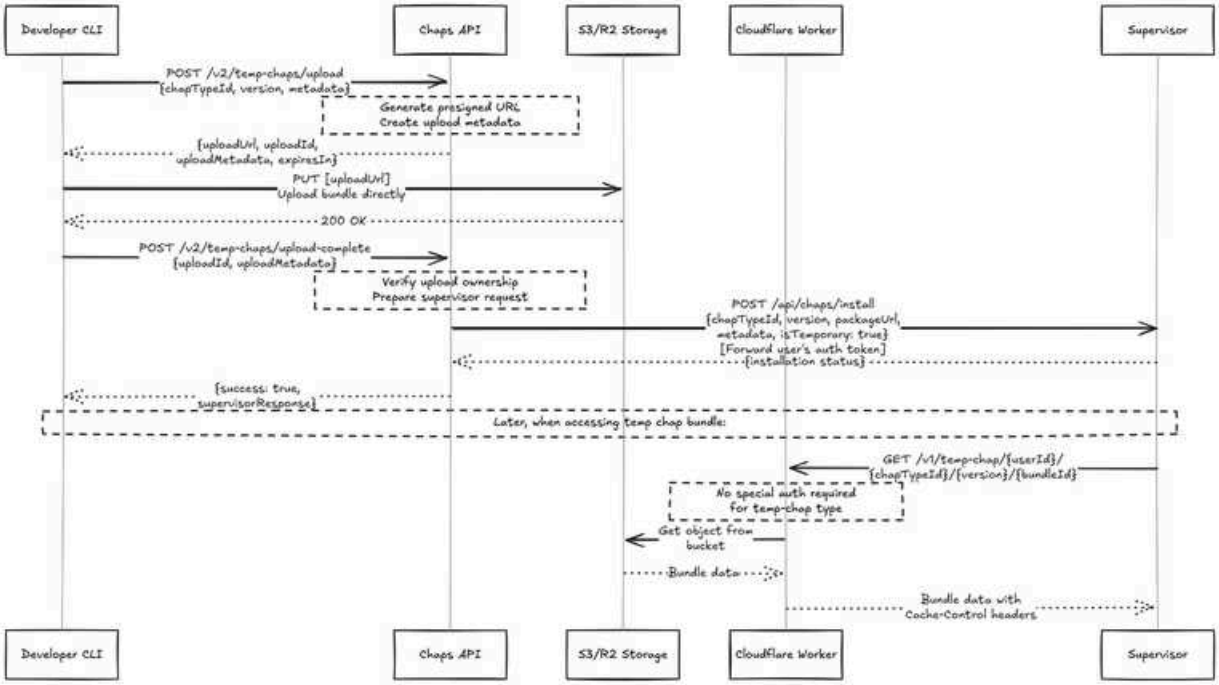
A chaplet needs to be able to download and install from a URL - code changes made.

CLI commands: `push`

I worked on `chap push`, which pushes a bundle to the chap registry. I will need to work with Levy tomorrow on the ACL / registry side. What I'm proposing is to change the tokens accepted on the registry for users to be able to create chap templates, not just the registry tokens. Chap bundles are assets, I don't want to break the other flows.

CLI commands: `install`

I've been working on a separate upload path for a chap, which bypasses the registry altogether, and allows a developer to upload their local bundle to their chaplet. Workflow below:



The process here is CLI requests an authenticated upload location (→ R2), uploads the bundle, then calls the API to signal a successful upload. When this happens, platform will do whatever post-processing is needed for a bundle, and call the chaplet directly with a download URL. Chaplet downloads bundle, chap is installed.

Post-processing: size check (max N Mbytes), any transformation needed *at upload time* from external-facing agent config to internal agent config, etc.

I'd rather not have to modify or write a CF worker for this - let's chat tomorrow Levy.

Authentication

To enable multiple cli commands to be run, we're going to have to save the user session token to the keychain on the local machine. For the install command, I'm asking for a new token every time - but nothing stops us from saving that token, and only requiring reauth when it expires.

Tomorrow

Prio: come up with plan for images.

Finish CLI install, then push.

We should also make a decision on how we move forward with search (I've merged it: https://github.com/lukilabs/chaplet/pull/87)

Jul 14., Mon

↑ Jul 14., Mon

Chaps Team

Peter Sipos

Plumbing around Chat Menu

- discussed some of the UX details with BalintO
- added support for overriding chap `settings` from requests (with message `metadata.settings`). While there dropped an older version of this feature which was to be able to send complete low level yamls in metadata, which we never did and anyway not implemented in the Typescript port.
- to control order of actions in lifecycle, added the same `sort_index` based sorting to lifecycle actions and user step conditions (what we have for context steps). This is needed eg when MCP setup completes, run the chat menu build first, then the setup complete message.
- related feature is to be able to properly await a `runPromptChat` action that goes to agent client service via socket (from lifecycle execution). Introduced a `runPromptChatResponse` message that comes back from agent client, and is awaited in gateway's agent client service. This way we can properly execute all actions on a user step change in a proper sequence as intended.
- also while testing, encountered and investigated some cli tool issues, which lead to Levy implementing parallel cli tool call execution

Chap manifest

I was also thinking several rounds on the high level chap composition manifest json, and actually leaning towards the generic array-of-versioned-modules approach (which I already started in the main branch already).

This will be a really flexible approach which will allow us to

- validate when a manifest is uploaded against currently supported modules in chaplet
- when upgrading a chaplet, keep support for earlier chap manifests and modules as necessary
- deprecate modules in the SDK, and warn developers that a particular chap feature or older version of the feature used by the dev, is going to be removed soon, and encourage upgrading to new version.
- enable different modules based on account or tier (premium features), or for staff (test or incubating agent features)

Next Week

Merge initial version of gcal calendar menu (Monday)

- it is already chap manifest module, so similar feature can be added to other chaps as well)
- this module is likely going to change a bit, when adding a more generic settingsDefinition module, but this is the beauty of this module approach, that we can just decide when to keep or stop support for this variant of the feature.

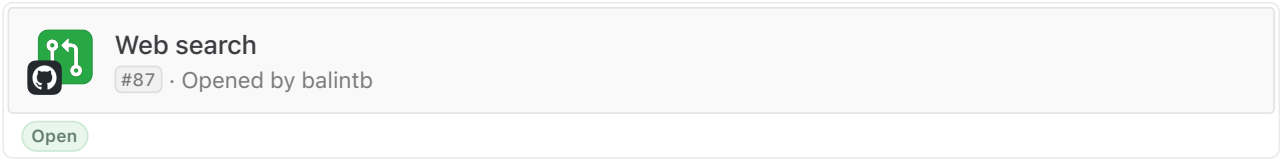
Have first version of the chap manifest API → the goal: can create a Github chap with purely the Chap composition manifest

Work on chap smartness

Balint Bartfai

Web search

Integrated EXA into Chaps today - *you should have received an invite into the org*. Chaps now have a web search tool.



However, so far I'm not convinced this is the quality we want to provide to users. I've been experimenting with different ways to prompt the model, it's fairly reluctant to use the tool at all. I have to specifically tell it to *search the web*.

```
                              search the web for places to eat gyros in budapest ❮
❯ [t] web_search
  I'll look up some popular places to eat gyros in Budapest for you.
  {"query":"best places to eat gyros in Budapest","numResults":7}
  ✔ "{\"success\":true,\"data\":\"{\\\"results\\\":[{\\\"title\\\":\\\"Üdvözöljük a
Leonidas Gyros
   oldalán!\\\",\\\"url\\\":\\\"https://leonidasgyros.hu/\\\",\\\"snippet\\\":\\\"#
Üdvözöljük a
  Leonidas Gyros oldalán!\\\\n\\\\nIsteni görög gyros közvetlenül Görögországból.
Kóstolja meg a
  valódi görög ízeket.\\\\n\\\\n![Tál gyros](https://leonidasgyros.hu/bolt.jpg)\\
\",\\\"score\\\":0
  .3023880422115326},{\\\"title\\\":\\\"Leonidas Gyros | Linkedin\\\",\\\"url\\\":\
\\"https://www.l
  inkedin.com/company/leonidas-gyros\\\",\\\"snippet\\\":\\\"Leonidas Gyros\\\
\nSlogan: A
```

Perhaps this is just prompting, but search results didn't seem to be good, OR the model just didn't pick correct ones. 3/4 places are not in Budapest.

```
| ❯ Here are some popular places in Budapest where you can enjoy delicious gyros:
|
|
|
|   - [Leonidas Gyros](https://leonidasgyros.hu/): Offers authentic Greek gyros with
flavors straight    |
|   from Greece.
|
|   - [ReGe Gyros](http://rege-gyros.hu): Known for its tasty offerings, located in
Pomáz, near          |
|   Budapest.
|
|   - [Paris Gyros](http://paris-gyros.webnode.hu): Another option not far from
Budapest, located in     |
|   Érd.
```

```
9  │
10 │   - [Gyros Kings](http://gyroskings-szodliget.hu): Offers a variety of gyros and
   is based in           │
11 │   Dunakeszi, close to the city.
   │
12 │
   │
13 │   If you'd like more details or want options specifically in the city center, let
   me know!              │
14 │
   │
15 │                          also find nice statues to sit next to that have an ice-
   cream parlour nearby ❮ │
16 │
   │
17 │
   │
18 │ ❯ Thanks for your suggestion! I've recorded your preference for finding nice
   statues to sit next       │
19 │   to, especially those with an ice-cream parlour nearby. If you have more
   specific details or          │
20 │   locations in mind, feel free to share them!
```

For any subsequent requests, no search was performed, only a promise to save it to preferences.

No web search was performed apart from one before the first answer below, and preference persistence seems a bit too eager:

```
1  │ ❯    Here are some places in Debrecen where you can enjoy gyros:
   │
2  │
   │
3  │   1. Ali Baba Gyros – [alibabadebrecen.hu](http://alibabadebrecen.hu)
   │
4  │      - Location: 22 Móricz Zsigmond út, Debrecen
   │
5  │
   │
6  │   2. Leonidas Gyros – [leonidasgyros.hu](https://leonidasgyros.hu/)
   │
7  │      - Offers authentic Greek gyros with real Greek flavors.
   │
8  │
   │
9  │   3. Régi Posta Étterem és Hotel – [regiposta.hu](https://regiposta.hu/)
   │
10 │      - While primarily a restaurant and hotel, they may offer Mediterranean
   dishes including gyros.    │
11 │
   │
12 │   4. Alshami Restaurant – [Alshami Restaurant
13 │
```

```
|   LinkedIn](https://www.linkedin.com/company/alshami-restaurant)
|
|      - Known for Middle Eastern cuisine, which may include gyros.
|
|
|   If you'd like more information about menu options or locations, just let me
know!                    |
|
|
|                                                                  how close is
alshami to LHDC airport? ❮ |
|
|
|
| ❯     I don't have information available about the distance between Alshami and
LHDC airport. If you    |
|   need this detail, please provide more context or specify the locations (such as
city or country),     |
|   and I'll do my best to assist you further!
|
|
|
|                                              lhdc is debrecen airport. how
close is it to alshami? ❮ |
|
|
|
|
| ❯    I haven't found any information added or updated regarding the distance
between Debrecen Airport  |
|   (LHDC) and Alshami. If you need details about their proximity or travel
options, please let me know, |
|    and I can help look up that information for you!
|
|
|
|                                                                search the web
for their proximity ❮ |
|
|
|
|
| ❯    I've created a new document to record preferences. If you need to track or
update any             |
|   preferences, you can now do so in this dedicated file. Let me know if you'd
like to add any specific |
|    details!
|
```

We definitely need to iterate on the prompting side a bit more - technically we have web search, but practically it's not great so far. I'd like to also try this out with some of the models with built-in web search tools - didn't get to this today.

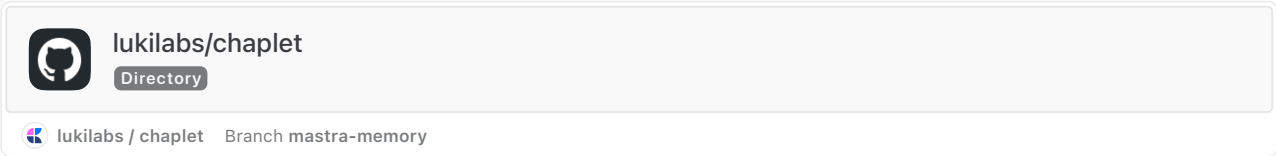
Mastra memory experiment & PoC

Started to experiment with Mastra memory. Currently, I'm using an SQLite DB for storing thread-scoped and user-scoped memory (`LibSQLVector`), with `fastembed` in-mem embedding (this would get much better with external - eg. OpenAI - embeddings).

Update: `fastembed` doesn't want to work on `arm64`. Switched to OpenAI embeddings.

DB file is currently hardcoded to `/data/mastra-memory.db`.

I wanted to try to **save** memory first. Next step will be to actually use it for recall. See branch:



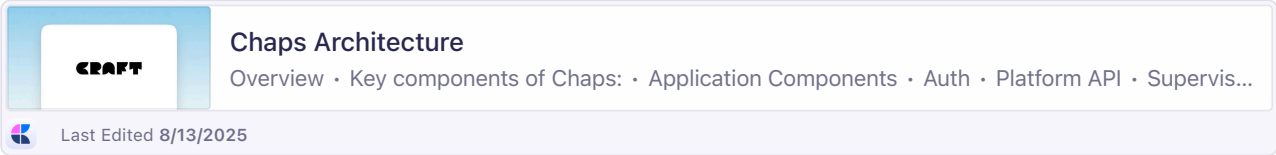
Compare: https://github.com/lukilabs/chaplet/compare/main...mastra-memory

So far, memory doesn't seem to be saved. The DB is opened, OpenAI embeddings are being created, but when opening the db file it seems empty.

I'll need to disable the memory on wakeups as it's not relevant and for now not working - there's no real user prompt to reply to, nothing to save into memory.

To sum it up: the concept seems great, I have yet to see it in action on the write side. My plan is to start running context creation in parallel somehow to be able to observe our context vs. how memory context - just simple logging or similar.

Architecture



Wrote quite a bit of documentation and sequence diagrams on Chaps architecture.

- Chaplet low-level ops
  - Chaplet boot
  - Persistence flows
- Auth flows
- Utilities

Next steps

- Developer CLI tool to bundle and upload chaps.

- Mastra memory continued from Friday's experimentation.

Levy Melnikov

API Fixes

Added `seq` parameter to the send message batch API for compatibility with other APIs. ⚠️ Let's not use it, it behaves differently in different APIs. It's already not used on chaplet.

- It's just an array index of the response in get messages
- It's just an array index of the response in send message batch
- It's an actual index of the message in the thread in send message ⚠️ For this we download the whole thread history, this is not scalable.

Made `metadata` in chap related APIs behave as a hierarchical JSON. Before it was a flat dictionary.

Grouped messages

Chaplets & Platform now support sending \ receiving batched messages. On chaplets images are not supported, they are silently dropped. I decided to first finalize the grouped messages support and handle images separately, as it turned out to be more complicated than I thought.

CLI Tool Fixes

With help from Schipy improved CLI Tool. Now it can batch CLI Tool calls and execute a batch of them in one go.

- This minimizes number of times that a sandboxed environment is setup. Instead of setting it up for each command, it sets it up for batches.
- Prevents commands to be ran in parallel

Github

Images Support

Tried to make my image handling API to work properly but stuck a bit. It works with the following hacks:

- Had to disable evaluations for runs that have images. For some reason evaluation errored out if the run had images.
- Had to cut out images out of communicator step. Currently communicator receives a json of the history. Since we base64 encode the image it extremely inflates that json that LLM can't process.

I tried making both the evaluations to work and communicator agent to "see" image as an image and not as a text string, but couldn't.

Balint Orosz

- Added Grouped Message Sending
- Improved Image sending UX
- Integrated & Tested Kimi K2 - It works pretty darn well
- Improved on Chap metadata
  - Chap icon is now either a SVG or inline Base64 encoded image. Both are quite small and we don't need separate asset management
  - Introduced Chap Styles - Chaps can define a number of colors (and later background image etc..) - this still needs to be displayed on the UI

↑ Daily Notes - 2025 July

Jul 11., Fri

Chaps Team
Peter Sipos · ℹ️ I did not know ahead (hectic week), but seems like I will have capacity to work half day Fri, 11 Jul. ·...

↑ Jul 11., Fri

Chaps Team

Peter Sipos

> ℹ️ I did not know ahead (hectic week), but seems like I will have capacity to work half day
> 🗓️ 2025. Jul 11., Fri.

Besides the discussions and planning we had, have 2 potential progress of interest:

Drafted around how the chap manifest could look like

Chap `composition.json` example (~chap app manifest)

```
{
  "mcpServer": {
    "id": "mcpServer@0.1.0",
    "name": "linear",
    "server": {
      "type": "remoteMcp",
      "url": "https://mcp.linear.app/sse"
    },
    "auth": {
      "type": "dynamicOauth",
      "resourceUrl": "https://mcp.linear.app/sse"
    }
  },
  "llm": {
    "id": "llmModel@0.1.0",
    "modelName": "openrouter(openai/4.1)"
  },
  "responseStyle": {
    "id": "customResponseStyle@0.1.0",
    "prompt": "Always use deeplinks for issue in the output..."
  },
  "settingsDefinitions": [
    {
      "id": "setting@0.1.0",
      "name": "teamScope",
      "label": "Active Team",
      "type": "singleSelect",
      "values": {
        "type": "mcp-gather",
        "serverName": "linear",
        "prompt": "fetch the user's teams",
        ...
      }
      ...
  ],
  "chatMenus": [

```

```
    {
      "id": "setting-menu@0.1.0",
      "setting": "calendarScope"
      "showEvenIfNoMultipleChoice": "false"
    },
  ]
}
```

settingsDefinitions examples

Model selector as a user facing chap setting

```
"settingsDefinitions": [{
  "id": "setting@0.1.0",
  "name": "llmMode",
  "label": "Chap Mode",
  "type": "singleSelect",
  "values": {
    "type": "static",
    "options": [
      {
        "label": "Fast",
        "value": "openrouter(openai/4.1)"
      },
      {
        "label": "Smart",
        "value": "openrouter(openai/o3)"
      }
    ]
  },
  "apply": {
    "type": "chapconfig-overlay",
    "chapconfigMapping": {
      "DEFAULT": {
        "llm": {
          "id": "llmModel@0.1.0",
          "modelName": "{{ value }}"
        }
      }
    }
  },
  "expose": {
    "toMainAgent": false,
    "toSupportAgent": false,
    "showInChapAdvancedMenu": false,
  }
}],
"chatMenus": [
  {
    "id": "setting-menu@0.1.0",
    "setting": "llmMode"
  },

```

```
]

```

Google calendar scope chat menu

```
{
  "name": "calendarScope",
  "label": "Active Calendar",
  "type": "multiSelect",
  "values": {
    "type": "mcp-gather",
    "serverName": "gcal",
    "prompt": "fetch the user's calendars",
    "refreshPeriod": "daily",
    "canBeRefreshedBySupportAgent": true,
    "additionalCachedDataPrompt": "cache the calendar id, user role, time zone for each calendar"
  },
  "apply": {
    "type": "prompt",
    "prompt": "Use the following calendar(s) as default scope for handling the user's request, unless explicitly told otherwise by the user in the particular message: {{ values }}"
  },
  "expose": {
    "toMainAgent": false,
    "toSupportAgent": false,
    "showInChapAdvancedMenu": false,
  }
}
```

scope selector chat menu

On more practical track, had a successful PoC run where, triggered on the gcal MCP setup complete, the agent collected calendars from MCP, and created a scope selector config from it, and also cached per calendar data.

Assistant: 📅 I have successfully identified that "calendar" is the appropriate scope for navigating and fulfilling user requests with the Google Calendar API. To make this process efficient, I have created a scope selector configuration that allows you to select from your available calendars. Here are the options you can choose from:\n\n- Holidays in Hungary\n- Gergő / Családi\n- Péter Sipos\n- Peter Sipos Work (PRIMARY)\n\nFor each of these calendars, I have also prepared a dedicated data summary file containing essential details, such as timezone, access level, reminders, and descriptions (if available). This setup will make it easier and more efficient to work with your calendars in the future. If you need access to a particular calendar's information or want to switch between them, the scope selector is now ready to use.",

```
Running command: echo '{
  "name": "gcal_calendar_scope",
  "metadataName": "gcal_calendar_scope",
  "options": [
    {"label": "Holidays in Hungary", "metadataValue":
"en.hungarian#holiday@group.v.calendar.google.com"},
    {"label": "Gergő / Családi", "metadataValue": "...@group.calendar.google.com"},
    {"label": "Péter Sipos", "metadataValue": "schipy@gmail.com"},
    {"label": "Peter Sipos Work (PRIMARY)", "metadataValue": "schipy@craft.do"}
  ],
  "isMultiSelectEnabled": false
}' > gcal_calendar_scope_config.json
```

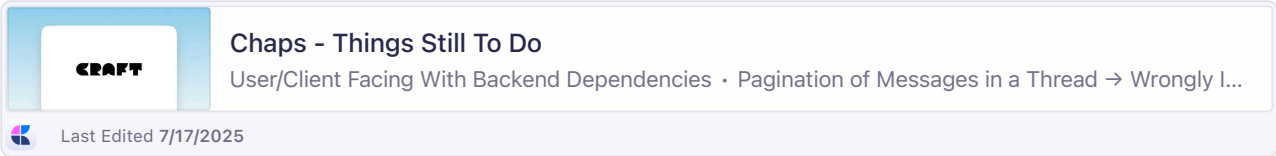
```
Running command: echo 'Calendar: Peter Sipos Work (PRIMARY)
timezone: Europe/Budapest
accessRole: owner
defaultReminders: popup (10min before)' >
gcal_calendar_scope_schipy@craft.do_cached_data.md
```

2025. Jul 11., Fri

would like to cuntinue a bit on the composition.json and scope menu feature support. Goal: merge a calendar scope selector feature into gcal chap (may not reach that tomorrow).

Balint Orosz

In the morning I worked on figuring out "the plan", so we can better align focus and distribute tasks before the beta.

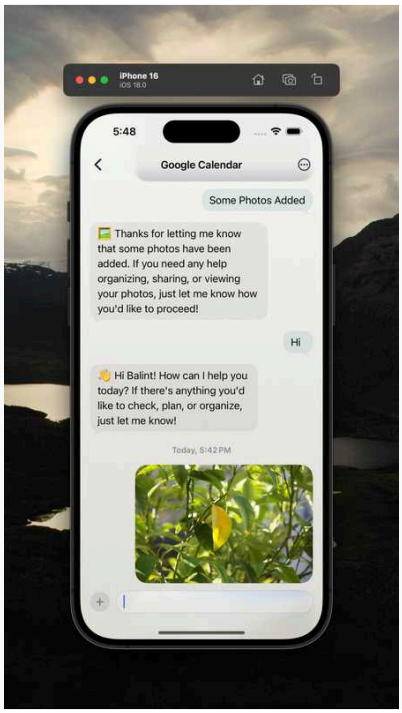


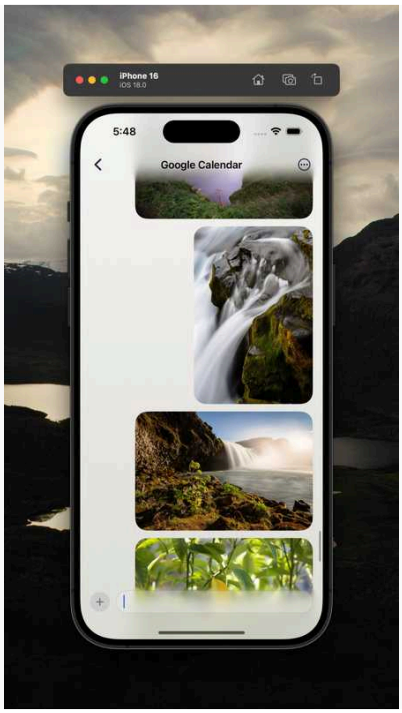
We than discussed this with the team

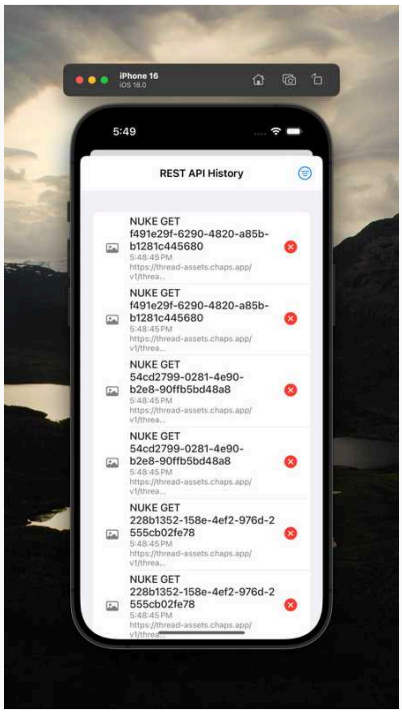
I than continued with pimping the app :

- Smoother pagination
- Improved Image Uploading/Handling (with MetalBlurHash as placeholders 🤘) cc
  n)
- Displaying the images in the app
- Logging image requests (Custom NukeDataLoader etc)

Tomorrow Levy will likely get the grouped message loading ready so than I can integrate and close the images area.







**Note : I also tried to get the "bouncy" scroll a'la iMessages app, but so far failed, but aint givin' up!*

Levy Melnikov

Message Grouping

I've implemented message grouping both on platform and on chaplet.

- Platform part is already on prod. One nuance to fix: allow not providing content if the message is an image.
- Chaplet part is still in the PR and is tied to the image handling stuff

Mastra Image Handling

I've created a PoC: https://github.com/lukilabs/chaplet/pull/84

It has following hacks:

- Had to disable evaluations for runs that have images. For some reason evaluation errored out if the run had images.
- Had to cut out images out of communicator step. Currently communicator receives a json of the history. Since we base64 encode the image it extremely inflates that json that LLM can't process.
  - ⚠️ I think this mistake might've cost a lot, I'll check our open router dashboard in the morning

Some learnings:

- Adding images increases processing time a lot. With a few images simple todo chap question is ~5 seconds, we'll need to figure out how and when to cut images from the history.
- I see in LangSmith that messages are marked as type: unknown. I don't know if it's a parsing issue or it really causes anything.

2025. Jul 11., Fri

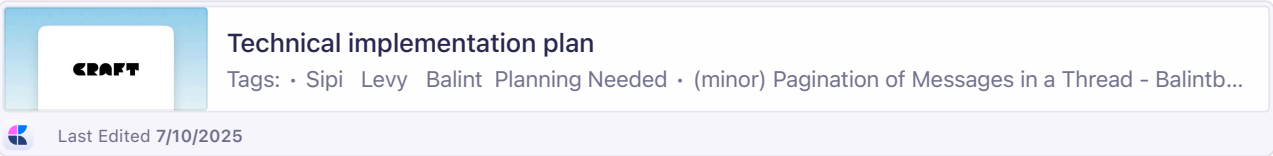
Finalize Message Grouping + Mastra Image Handling

Balint Bartfai

Planning from a technical and work stream perspective of "***The plan***" (as Balint put it).

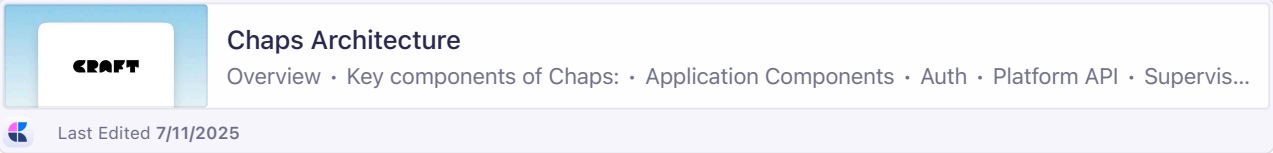
> *Scheduling has been deprio'd until a later stage, see plan for timing on this.*

Technical and delivery roadmap:



Architecture

Started work on consolidating the architecture docs, so we can have an overview of all the moving parts, and where they are in the codebase:



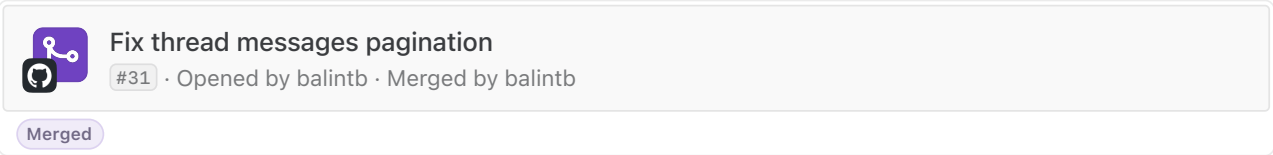
• • •

BetterStack has been upgraded to a paid plan for 34USD / month. 30 day retention for logs.

We are on the monthly, pay-as-you-go plan. Once we have a better idea of the volume, we'll switch to prepaid credits and yearly billing, saving us at least 20% vs monthly, and even more with the proper volume discounts.

• • •

Fixed the pagination bug. Problem was trivial, took a little investigation to find the root cause. Added comprehensive test suite to ensure we don't run into something like this again.



As a general rule, again, please:

> For any bug fix, any new core functionality (for example, scheduling) ensure there is a test suite. **Unit tests for service logic at a minimum, integration tests for critical workflows.** All systems have mocking in place for dependencies. It's mostly integrated into the Github builds. We use `bun:test` for the framework. Adding tests is straightforward and ensures breakages are kept to a minimum by enforcing contracts *at least on the service level*.
>
> This might also be straightforward, but for further safety:

- when accepting multiple parameters for a method, **always use composite, typed param objects**. All of our IDs are strings, it's **extremely** easy to mix up the order of params passed, this will not be caught by the compiler. Explicitly calling out the param object properties ensures we're not tied to the order of params passed, and prevents these types of bugs.
- decouple abstract implementation from the tech-specific implementation. For example, instead of having a CacheService tied to a Redis instance it instantiates, have a generic CacheService with a Redis adapter. Dependency injection makes it testable.

Tomorrow

- Continue as per tech implementation plan
  - Expand architecture docs
  - Web search setup
  - Start implementation of CLI for chap management for devs. See how this fits into plan.
  - If I have time: Kick off experiment in Mastra for memory. My plan is to
    - Understand how Mastra implements this
    - What persistence options we have - local file, remote persistent store, etc
    - Start testing with agents

↑ Daily Notes - 2025 July

Jul 10., Thu

Chaps
Levy Melnikov · CLI Tool · I've implemented a new version of CLI tools that uses chroot env -i to remove env variable...

↑ Jul 10., Thu

Chaps

Levy Melnikov

CLI Tool

I've implemented a new version of CLI tools that uses `chroot` `env -i` to remove env variables. It doesn't need root access and can be run on k8s without privileged mode.

API Fixes

- Fixed supervisor to properly forward non-json payloads. This time I tested everything and hopefully now this will work and won't break anything.
- 🤦 I didn't confirm my email with namecheap so `chapsdevng.com` was deactivated which lead to chaplets being not available \ chap creation failing. Now I verified my email and it's back online. **Before release we should buy a domain from the company account.**
- Looked at image upload API, but couldn't find anything wrong. It worked for me. I added a ton of logs there so if it fails, I will be able to debug it easily cc Balint Orosz

Next Steps

Look into images in chaps. So far I see at as:

- Allow messages to be asset only
- Allow multiple messages in `/sendMessage`
- In case multiple messages arrive in `/sendMessage`
  - Add a generated `groupId` to all of them
  - Send them **in one event** to chaplet. This is required so that one message group with multiple messages(for example text + image) doesn't trigger multiple runs.
- On chaplet side:
  - Allow unwrapping multiple messages from a single event
  - Correctly pass images to LLM

Peter Sipos

Had a quite ad-hoc day, half of it was various support/investigation topics:

- reviewed **scheduling** PR of BalintB, we had a call as well discussing the solution and details
- researched **grouped messages vs message parts** a bit, see Slack thread
- debugged various issues BalintO encountered (and eg switched to unsafe cmd tool for now)
- explored the options to have an **address book**, which BalintO identified as a UX bottleneck in gcal chap, we will have a discussion tomorrow morning about this topic
- did some quick widget cleanup

In the other half of the day I wanted to progress with something, and since did not yet have the chance to discuss agent memory exploration stuff with BalintB (we were still discussing scheduling), I did some **plumbing on the chat menu support**, which is a similar UX bottleneck as the address book. I'll likely shelve it tomorrow to return to more unclear exploration topics (we will discuss prios and work split) but I already have some progress on chat menu, which will be a good head start next week to deliver it (seems likely within a day now to have the dynamic scope selectors, like Linear project or GCal calendar, static menus will be a follow-up topic)

 Balint Orosz

Bit of a screwed day, most of the things I started ended up in discovering bugs in the underlying platform, so kept jumping between the topics.

Did manage to add a different visual for status messages to the app, background data fetch and paging to the chat view, but I.e paging is still quite weird as the backend doesn't do it well.

Balint Bartfai

Scheduling, wakeups

Spent most of the day working on getting the agent to schedule wakeups.

Scheduling works by providing the agent a tool to create a wakeup at a timestamp, with instructions on what to do when it wakes. When waking up, it can schedule the next wakeup, and the process continues if the user asked for ie. a recurring notification.

Sounds straightforward, but it turned out to be deceptively simple. Couple of issues that cropped up, discussed with Sipi

- keeping initial user intent intact is needed. The agent can't keep on giving instructions for the next wakeup - relying on the previous instructions alone, by modifying it slightly every time, the Nth wakeup will drift too far from the original, this is not viable
- relying on a fixed message ID as the original user intent doesn't work for a number of reasons
  - even though the user expressed their ask for a notification / reminder, there's no guarantee the agent actually schedules a wakeup *based on that message alone*. There could be multiple back-n-forths between user and agent, after which the agent schedules the wakeup. There's no way to associate a message ID with the original intent, or if there would be, this would have to be marked by the agent - more tool calls. Another way would be to take the whole message history when waking - but this way we spend too much tokens, and there's no guarantee the intent can be determined from whatever fits into the context window.
  - if an agent wakes, it doesn't do so based on a user message. Again, this means that an original intent must be stored, as opposed to a message ID it responds to each time
- saving the original intent - due to the multiple messages derived intent problem described above - can only be done by saving whatever the agent describes as such, like a summary of what the user asked. To solve this
  - we can instruct the agent to always keep the original user intent - which is has set on the initial wakeup's metadata - intact, and in a separate property, describe what the next wakeup should do. For example:
    User: `when the weather is rainy, remind me the next day to wash the car`
    Original intent: (as above)
    Agent schedules wakeup for the next day to check weather:
    Next wakeup instruction: `check if the weather is rainy`, OR, `remind the user to wash the car` (if the weather was rainy on that wakeup)
  - enforce an immutable original intent in the wakeup metadata, so the agent doesn't pass the original intent in a modified form (which, from what I've seen, does happen). The agent will still try to override the original, but on wakeup we ignore whatever the agent says, and place the original intent back to metadata
- an agent can schedule multiple wakeups from a wakeup. This means we can't really rely on the original intent of the wakeup to pass to any wakeup scheduled inside a wakeup LLM run. We could potentially pass original intent on the first scheduled wakeup (first scheduling tool use) - but there's no guarantee of ordering

A technique we could try is introducing the context of "wakeup threads". A thread starts on the first wakeup scheduled, and the threadId propagates - via prompting - throughout the wakeup chain (ie. recurring events). I'm not convinced this would be exact every time - we'll have to come up with use-cases, edge-cases we can eval the behaviour on. The current thinking is around asking the agent for an original intent unmodified, and relying more on the next wakeup instructions, asking for a "state" of the original intent (see example above).

Underlying tool creating mechanism had to be changed to have thread context for the wakeups - as the agent has no concept of what thread it's in, etc. Wakeups are tied to a thread.

This still requires experimenting. I've underestimated the ambiguity in behaviour here, even though the underlying mechanism requires multiple API calls and is *slightly* complex, lots of moving parts, but deterministic.

Interview

Conducted interview for the Agent Quality Engineer role.

Tomorrow

- Finish wakeups based on the immutable *original* intent + more detailed next wakeup instructions concept above. Estimating another day (=tomorrow) on this to iron out behaviour.
- Memory planning with Sipi

↑ Daily Notes - 2025 July

Jul 9., Wed

Chaps Team

Peter Sajevics just ran into this reddit post and I thought it could bring some inspiration: · Image · Peter Sipos · Esti...

↑ Jul 9., Wed

Chaps Team

Peter Sajevics just ran into this reddit post and I thought it could bring some inspiration:

I Tried 50+ AI Tools for Work and Marketing - Here Are My Favorites

I spent the last few weeks testing over 50+ AI tools to find out which ones really save time and help with work and marketing. Here are the tools I use the most:

1. **ChatGPT:** Helps me write emails, brainstorm, and summarize stuff fast.
2. **Google Veo:** Makes videos from prompts. Great for quick marketing videos.
3. **Scribehow:** Turns screen recordings into step-by-step guides.
4. **Clay:** For research and data enrichment.
5. **Canva:** Easy way to make social media posts and graphics.
6. **Notion AI:** Plans my day and takes meeting notes automatically.
7. **Otter AI:** Record and write out what's said in meetings.
8. **Grammarly:** Checks my spelling and grammar quickly.
9. **Copy AI:** Write social posts and ads in seconds.
10. **Hunter IO:** Finds email addresses for outreach.
11. **TinyPNG:** Makes images smaller so websites load faster.
12. **Proofademic:** Checks if my writing looks too AI-generated.
13. **Perplexity & NotebookLM:** Fast tools for research, learning, and finding answers.
14. **Dia Browser:** Lets me search and organize info from the web quickly.
15. **N8N:** Helps automate tasks and connect different apps without coding.

These are the tools that really help me get work done faster and better. What tools do you use every day? Share your favorites below!

Peter Sipos

Estimated input token count of MCP tools

token count of gcal tools is ~1.6-1.8, plus we have some own tools as well, eg cli tool is +168, total is ~2k

linear tools token count is also at least: ~1.8k

I approached in both directions

- based on the the minimum total string content in the tools (which I extracted manually from google MCP repo, and for Linear from MCP inspector connected to Linear)
- and also checked a LangSmith run and substracted message array tokens (1050) from total input tokens it reported (3177), where the difference ~2.1k tokens which is the not visible tools related tokens

I think we can roughly go by the estimate that **currently MCP tools for agent is 2k input tokens**

Side note: this opens up an interesting aspect to look into LLM based api compression, where we once ask the LLM to compress the MCP api (eg extracting duplicate explanations from across different tools - same/similar parameters explained at multiple tools, drop unnecessary/obvious descriptions, etc), maybe we can win here some serious buck, as it seems to me many of the tool descriptions in Linear and GCal api is quite unnecessary (like "creatorId: Filter by creator ID" - stating the obvious)

for reference:

TXT

gcal_mcp_tool_string_dump.txt
TXT Document
6.0 KB

linear_mcp_tool_string_dump.txt
TXT Document
7.0 KB

Memory solutions

> I did a research on agent memory, my current bet is to start with
>
> - for thread history: `Mastra's` built-in memory solution (very quick start, not a big issue if we need to swap it out later)
> - for long term fact/user memory: roll our own `LLamaIndex` storage (flexible solution that scales with needs)

Premise / what we need:

history:

- store it inside chap (part of "chap state")
- per thread storage/query
- be able to limit to model context size
- potentially semantic search in past messages
- separately handle tool calls vs actual thread messages

This is kindof exactly what Mastra's thread scope memory does → it seems to be the fastest time to market, and if we need to drop it later in existing chaps (swap solution for something else), it's no big deal, as we can always fall back thread history coming from realtime platform.

we can use its MemoryProcessors to quickly implement

- safe guard to fit into model context window
- manage tool calls with different TTL then regular messages

long term fact/user memory:

- store it inside chap (part of "chap state")
- **have the option to semantic search** within memory items, this is important to be future proof, especially as pressure to reduce input token count increases to control costs.
- **also support tag/label based storage/retrieval**: this could be our initial control point on the whole storage and retrieval strategy.
  - we already have different scopes in mind that we want to manage, eg
    - "gcal-api"(mcp api learnings)
    - "people" (a chap type specific important concept, like in calendar)
    - "gcal-api-data" (cached data from mcp server)
    - etc.

If an agent is provided gcal MCP tools, then we can retrieve gcal-api tagged learnings, but if not, we won't include it.

- tags/categories could be what chap devs can describe under memory options:

  > This chap should remember useful facts and knowledge extracted from conversations about the following topics:
  > *(example config, provided by chap authors)*
  > **people** - identity and contact information of people the user shares calendars with
  > **event-defaults**: useful information about creating new events (like location, reminders, preferred times etc)

  plus we include built-in tags, eg related to MCP servers, and also could add a *misc* for non-categorized

- later we can run cheap & fast classificator model on user message to decide what tags are useful to include in answering the request, eg if user is asking about her daily schedule, then we don't need to include `people` knowledge (which is more for create/update event actions)

I checked a couple of solutions out there for such a long term agent memory - here we also have to be more cautious as if we change the solution, we will likely need to migrate existing chap memories.

Mastra built-in: not good enough in terms of feature set for the above memory ❌

LangMem: nice, but **only Python SDK** ❌

Zep: very promising, but earlier available self-hosted edition is now deprecated, so it's **an extra cloud service**, and chap's memory would live on 3rd party ❌

LLamaIndex-TS: flexible, has TS SDK, supports both semantic vector search if we want, or just tag based retrieval, stores in local files. ✅

Example LLamaIndex setup
Snippet

• • •

`merged` Some tweaks to our message FrontMatter (help prompt caching)

message FrontMatter tuning

- discard humanized dates (eg "x minutes ago") as they change the message over time busting prompt caching
- port actual old sender type logic, which omits unnecessary actual user id for now, as long as we're in single user chat
- explain last date is the current date

Balint Bartfai

Wakeup on chaplets cont.

Chaplet wakeups were not as simple to implement as just an API call to the chaplet.

First, tools had no context whatsoever about where they were being run.

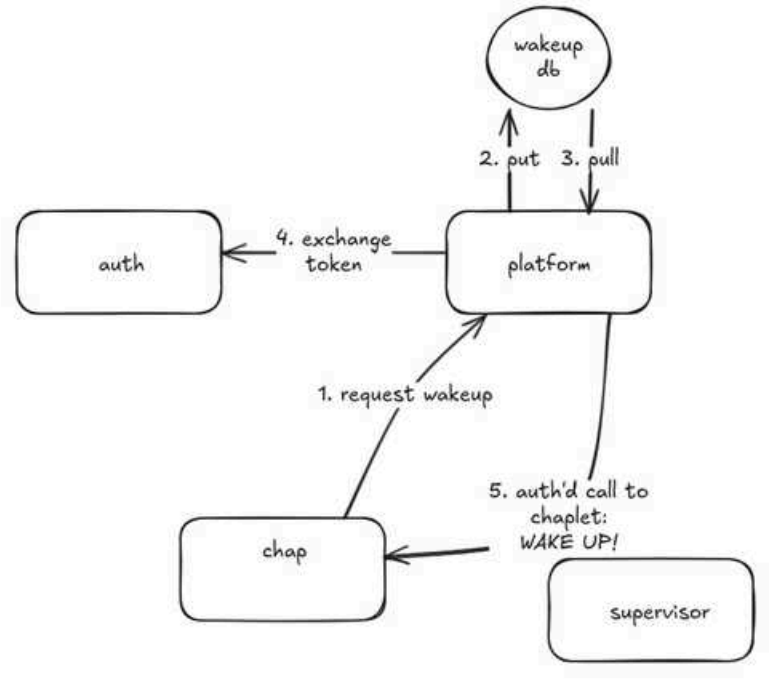
The chaplet then has to make a call to Platform API to request a wakeup. This call - as is all other calls from a chaplet - is authorised with the chap's session token. Chaplets can be routed requests to based on the auth token's user ID - otherwise the supervisor does not know which chaplet is responsible for making the call. Because platform only receives the chap token, it has to somehow grab a user token when making the actual wakeup call.

To solve this, Auth can now generate a short-lived user session token from a chap token. It does this right before calling the chaplet's wake up lifecycle event, triggering the agent.

The full process is now

1. Agent → Platform API (Scheduling)
    - Agent uses `schedule_wake_up` tool (and delete, list, etc. tools later)
    - Gateway on the chaplet calls platform API
    - Platform API stores wakeup with auth token in Redis
2. Platform API Processing
    - Processor moves due jobs from scheduled to processing queue
    - Consumer picks up from list, then
        - Retrieves auth token from Redis
        - Exchanges chap token for low TTL user token
        - Calls supervisor → chaplet with `wake_up` lifecycle
3. Chaplet receives `wake_up`
    - Gateway handler receives run-lifecycle POST
    - Calls lifecycle execution, which then
    - triggers any configured `wake_up` actions (now it's `run.prompt.chat`)

Visually



Show me the code!

Auth:

Chap token exchange for user token
#38 · Opened by balintb
Merged

Platform API / running jobs:

Scheduled jobs
#30 · Opened by balintb
Open

Chaplet lifecycle events WIP:

Wakeups lifecycle events WIP
#80 · Opened by balintb
Open

Tomorrow

- Lots of E2E testing on this. Chaplet part is still WIP, don't understand a lot of the config stuff going on - I'll chat with Sipi.
- Discuss Memory as well with Sipi

Balint Orosz

Mainly was getting images to work (well at least on the UI and figuring out how we want to eventually make them work).

We have iMessage style image attachments - we can have multiple ones, scroll, remove, add more animate etc. Overall lots of details.

On the architecture side :

- We have this trick that from the interaction perspective the user might want to do operations on individual images themselves, so on a data model it makes sense to separate an image as a standalone message
- However the LLM needs to get the entire message together with images
- For this I recommend introducing something as a messageGroupId or similar so we can logically group these, but still have atomic messages we can later easier use to handle stuff.

Levy Melnikov

API Fixes

- Supervisor now properly forward non `application/json` calls, meaning that update file in a chap should now work

Fixing cli tool

CLI tool stopped working on new K8S cluster, because **bwrap doesn't have permissions to create a sandbox for the command**. Fixing this turned out to be more difficult than I anticipated.

Option 1: Run containers in privileged mode: This way containers will have root priveleges and thus bwrap will be able to create sandboxes. It also means that in case of a breach(if bwrap's sandbox is bypassed somehow) a container will not only have unrestricted access to all of container's data(user of that chaplet), but will also be able to access other containers(other chaplets). I think this is a no go: containers \ k8s provide a lot of isolation, bypassing it with privileged mode for the sake of having our own implementation of sandboxing via bwrap doesn't seem ok.

Option 2: Use VM container runtime

Normally a container shares a kernel with the host node(that's why using privileged mode is a bad idea). But there are newer container runtimes that run completely isolated virtual machines. With these virtual machines even if we give all root level permissions to the container, breach won't be able to spread to the host or other container, because they are completely virtual.

One such runtime is kata containers + firecracker. Unfortunately we can't use it with current Digital Ocean setup, because they don't allow to change the container runtime. We could migrate to AWS for example that does support it. The issue here is that for this to work host has to have virtualization, meaning that it has to be a bare metal server, at AWS that's ~$2k\month for a minimal node. And on top of that we'll need to move yet again.

Long term this might be practical in case we:

- Will stay with chaplet=stateful container approach
- Will introduce more unsecure components like custom client code MCPs that run inside of the container

Option 3: Simplify sandboxing

Currently we use bwrap for sandboxing. I'd suggest to:

- Rely on container \ k8s isolation for blocking dangerous operations(syscalls, `dev` access)
- Use a simpler approach for:
  - Stripping env variables with `env -i`
  - Isolating to `docs` folder with `chroot`

This approach doesn't require root level access. I'm already working on this version and will finish early on Wednesday.

```ts
// 0.  Install the deps you'll see here
//    npm i llamaindex @llamaindex/openai @llamaindex/chroma

import { Document, VectorStoreIndex, storageContextFromDefaults } from "llamaindex";
import { ChromaVectorStore } from "@llamaindex/chroma";
import { Settings } from "llamaindex";          // so we can set the embed model

// -- wire up an embedder (OpenAI, Ollama, etc.)
Settings.llm = { model: "gpt-4o-mini" };        // or Settings.embedModel = …

// ---------------------------------------------------------------------------
// 1.  Boot (or reload) the index on local disk
// ---------------------------------------------------------------------------

// choose a folder — everything ends up under ./mem by default
const persistDir = "./mem";                     // ship or mount this file/folder

// Chroma gives us metadata filters out-of-the-box and can run embedded
const chroma = new ChromaVectorStore({ collectionName: "agent_memory" });

const ctx      = await storageContextFromDefaults({
  persistDir,             // where *doc*, *index* and *vector* stores live
  vectorStore: chroma,
});

// If the directory already contains state we'll get it back automatically
const index    = await VectorStoreIndex.init({ storageContext: ctx });  // creates empty
//        └─ could also load an existing index by ID if you have several

// ---------------------------------------------------------------------------
// 2.  Helper: write a new fact (Mastra tool would call this)
// ---------------------------------------------------------------------------
export async function writeMemory(text: string, tags: string[]) {
  const doc = new Document({
    text,
    metadata: {
      tags,                               // arbitrary array
      ts:    new Date().toISOString(),    // ISO timestamp for future TTL jobs
    },
  });

  await index.insert(doc);                // upserts into Chroma + doc store
  await ctx.persist();                    // flush to ./mem [oai_citation_attribution:0‡docs.llamaindex.ai](https://docs.llamaindex.ai/en/stable/module_guides/storing/save_load/)
}

// ---------------------------------------------------------------------------
// 3.  Helper: pull N facts by tag OR semantic sim (prompt hook)
// ---------------------------------------------------------------------------

```

```
export async function loadContext(opts: {
  tags?: string[];
  query?: string;
  k?: number;
}) {
  // tag filter (cheap lookup — no embedding)
  if (opts.tags?.length) {
    const qe = index.asQueryEngine({
      preFilters: {
        filters: opts.tags.map(tag => ({
          key: "tags",
          value: tag,
          operator: "contains",          // any doc whose tags include <tag>
        })),
      },
    });                                  //
[oai_citation_attribution:1‡ts.llamaindex.ai](https://ts.llamaindex.ai/docs/
llamaindex/modules/rag/query_engines/metadata_filtering)
    const res = await qe.query({ query: "" }); // empty query ⇒ just filter
    return res.sourceNodes().map(n => n.text);
  }

  // pure semantic search when no tag given
  const retriever = index.asRetriever({ similarityTopK: opts.k ?? 5 });
  const hits      = await retriever.retrieve({
    query: opts.query ?? "",
  });
  return hits.map(h => h.node.text);
}

// ---------------------------------------------------------------------------
// 4.  Tiny demo (delete once it's wired into Mastra)
// ---------------------------------------------------------------------------
await writeMemory("Peter Sipos loves Ethiopian espresso.", ["people", "coffee"]);
await writeMemory("MCP API v3 deprecates the /login route.", ["mcp", "api"]);

console.log("--- by tag 'people' ---");
console.log(await loadContext({ tags: ["people"] }));

console.log("--- semantic: 'espresso' ---");
console.log(await loadContext({ query: "espresso" }));
```


Jul 8., Tue

↑ Jul 8., Tue

Chaps Team

Balint Bartfai

Planning in the morning. Goals for this week:

- Mastra fully integrated, Orchestra sunsetted (Tue. morning)
- Get rid of ambiguity around how we solve
  - Memory
  - Scheduling
- Client cosmetic configuration

Mastra Workflows

We found some issues with the current implementation of agent configs and workflows in Mastra. Context wasn't being passed along properly and branches weren't being run in the workflow → no runtime errors - which was not true.

Rewrote the workflow-in-workflow approach to a simpler step-by-step one

Fix mastra workflow
#78 · Opened by balintb

Merged

Scheduling

Planned the week in the morning, I took on scheduling.

Chaps need to be able to "wake up" on certain events or a schedule. We had discussions about cron syntax or even simpler schedule definitions, but agreed that it wouldn't be flexible enough, and it wouldn't feel like an intelligent AI agent.

How it works

To keep things simple initially, I've decided to build the system out in redis. Other serverless queueing services could also work, but we have the Platform API running, we have redis, let's keep it simple for now.

- A job request hits the API (via the agent's tool call), with a source message ID (where the user asked for it / chap told it)
- The job, along with metadata, gets added to a Redis sorted set, with the score being the timestamp it will run at
- Periodically, a service looks at the sorted set and retrieves jobs that should be running (O(1) op)

- Atomically places them in a redis list in one transaction
- The list is then picked up by consumers, who handle execution

When being woken up, an agent will decide when they next want to be woken, and use the scheduling tool to schedule the next timestamp. This way, only the next wakeup is kept in redis, ensuring there's a space upper bound.

Whats still needed:

- Agent side tooling so the agent can request a 'wakeup'
- Chap API endpoints for the webhooks

Implemented the infra on Platform to manage scheduled jobs:

Tomorrow

- Finally merge mastra and nuke the orchestra code from chaplet repo
- Finish scheduling: **agent tooling, chap endpoint**

> **Please note:** I will have very limited availability outside of normal working hours this week, as I'm packing, transporting, moving place every day after work until very late.

Levy Melnikov

2025. Jul 7., Mon

Issues

- ✅ Chaplet ran out of RAM. I increased limits 500MB→1000MB
- ✅ Auth prevented chaps from calling APIs → fixed
- ⚠️ bwrap doesn't work on k8s. `Result from 'run_command_safe': "bwrap: Failed to make / slave: Permission denied\n"` . bwrap - is the command that we use to for a sandboxed cli in the cli tool. It seems that it doesn't work out of the box on k8s.

API changes

Now all chap related APIs have `metadata` field in auth. cc Balint Orosz

- You can set it on creation and update later via new `PATCH /api/v1/chaps/:chap_id` API
- List and get APIs now also return this field
- Field itself is an arbitrary key-value dictionary

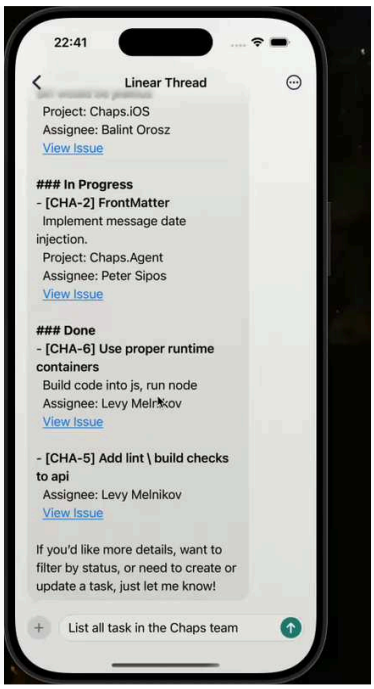
Auth: Chaplet creation

- Now if supervisor returns a `CHAPLET_NOT_READY` error it will be forwarded into response of auth
- In auth on login(technically on every token renewal) we call an API on chaplet to trigger it's creation

Push notifications

~~Started looking into notifications, but nothing tangible yet~~

▼ Basic infra is in place, demo video



Creandum Meetup

I attended First Monday Meetup by Startup Hungary. I met Marci and Luca there and we talked a lot. He seems to be super excited to join!

2025. Jul 8., Tue

⚠️ First thing is to fix `bwrap` so that cli tool works again

Next: finish push notifications

> Currently it works in the following way: when user is not connected with a websocket → they will receive messages via push notifications.
>
> Refinements:
>
> - Title of the push should be the title of the thread \ name of the chap
> - Deeplink should lead to the message or at least to the thread

I have this PR . I set up permissions for the push notifications and request device token on the startup. It has to be passed to the backend once the user authorized. I didn't implement that.

Stretch: Continue with Stripe

Balint Orosz

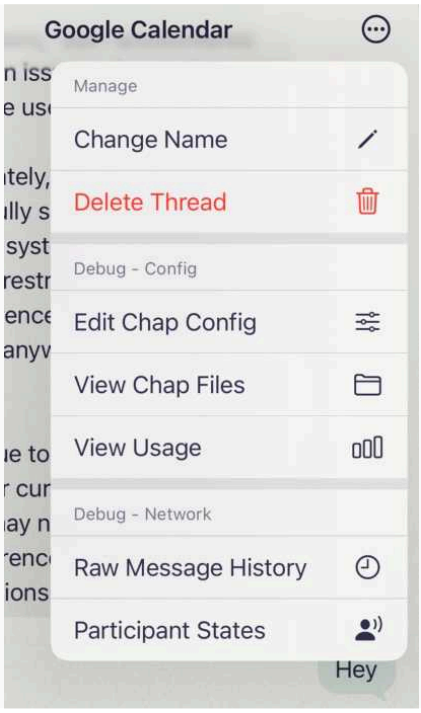
Goal for the week :

- Have a very smooth e2e flow from installing the app to registering to starting to use the first chap.

This practically means that I want to focus a bit more on the store, how chaps are presented, how they are installed go back to the login screens etc.

In the morning I did the light/dark mode implementation, as we didn't quite support that before. I also fixed some smaller bugs and glitches.

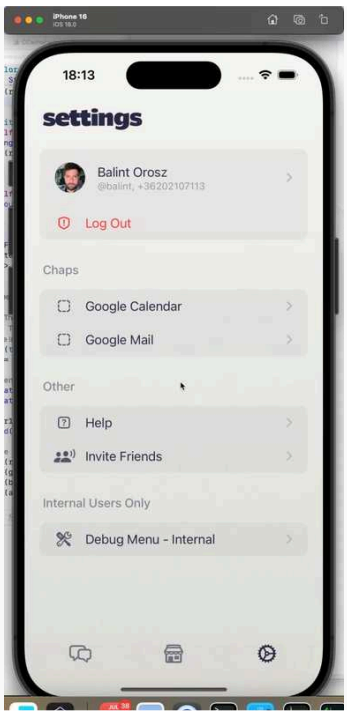
One interesting thing I did is I could actually Swizzle UIColors, so system components now use my colors - might be interesting for Styling and some visual flare in Craft as well cc Peter Adam Wiesner Anna Barta Christoph Locher



The label of the UIMenu is slightly blue-ish

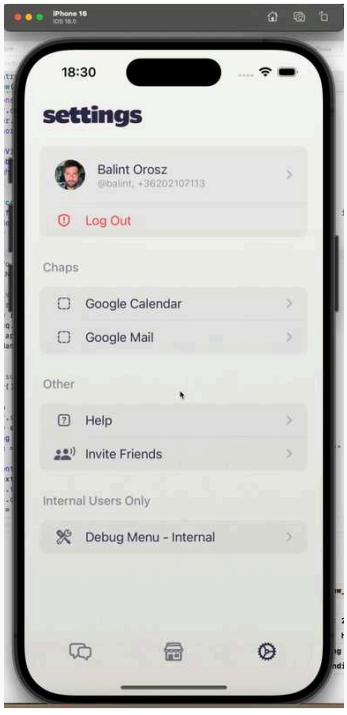
Update : This can get a wee bit messy when light/dark mode changes

I also added in the file endpoint so we can inspect what files the chap has :



But couldn't execute the update as we have permission issues.

I also added the price / usage endpoints so we can easily see how much each requests cost - I can tell you - too much 🙂



I will wrap up tomorrow image sending (so we can also send images to the LLM), and move on storing metadata for Chaps / pimping up the store / chap details screens.

Peter Sipos

> General sentiment: all in all I start to feel the stressful clash between the deadline and the huge amount of stuff that's still just planned (actually, since a few weeks already). I mean the deadline is good, we're already quite late compared to early plans. We (at least I) did have to spend another day on just Mastra migration, but at least scheduling capability is now planned, cleared and looking simple (one dark horse down from backlog).

Was still spending much of the day on fixes on Mastra branch to so that we can merge with everythinf working as before, some examples

- merged tool call user facing commentary support (the prop added to tool calls is named `messageToUser`)
- some leftover fly.io cleanup, as saw errors in log
- as we had our users recreated in new infra, fiddle a bit in local env to env the new user and be able to test. Also had a few rounds setting up env for all the latest services to have everythign running.
- eliminate some compile errors in our Mastra code (mostly around workflow execution), found an issue where skip communicator was not working ok, as both side of the branch (with and without) executed (all branch conditions get evakuated, and matching steps run parallel in Mastra)
  - BalintB had several merge conflicts later as I was already pushing much of the fixed on main, a little comm mishap, but I resolved the conflicts
- during testing realized that Mastra prefixed tool names with the mcp server name, eg list_issues became `linear_list_issues`, as these tool names are references in agent configs, had to update these
- and the tool interface itself also changed with the migration, and I realized that our MCP call based context enrichments silently failed (their Python counterpart where it was ported from was too robustly written) so had to fix up how MCP tools are programatically called and their result is extracted.

- then I had an inconsistency in the mcp configs `name` vs `server_name` which caused the gcal chap tool names to be prefixed with `undefined` - fixed

My Mastra merge checklist:

- [x] ~~functionality~~
- [x] ~~tracing~~
- [x] ~~compile issues / PR technical polish~~
- [ ] evals? - I have to talk with BalintB what this is about, now it we have evals running for all requests, which are also intertwined eg in LangSmith tracing
- [ ] quick sanity check regarding agent interruptability

Scheduling

We also discussed scheduling with BalintB, more details in his daily. I think we cleared ambiguity and it is now in the straightforward feature zone.

2025. Jul 8., Tue

Have to sync with BalintB as after Mastra merge I was kindof planning the agent side of scheduling, which I see he also planned.

Otherwise

- start planning around history (eg compression) & "user memory"
- check a bit current cost shape
- look into / implement agent interruptability

↑ Daily Notes - 2025 July

Jul 7., Mon

Chaps Team
Balint Bartfai · Worked on sending traces to BetterStack. If this would work, we have full visibility of the whole proce...

↑ Jul 7., Mon

Chaps Team

Balint Bartfai

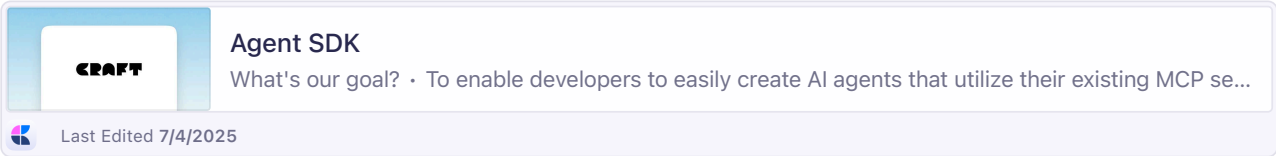
Worked on sending traces to BetterStack. If this would work, we have full visibility of the whole process. As traces contain a traceID consistent with all our other systems, we can end-to-end, including the exact LLM calls, trace a user request. Code is in place but it's not showing up in BetterStack for some reason - I'll need some help with this Monday Levy Melnikov .

Branch:

lukilabs/chaplet
Directory
lukilabs / chaplet Branch betterstack

Agent SDK

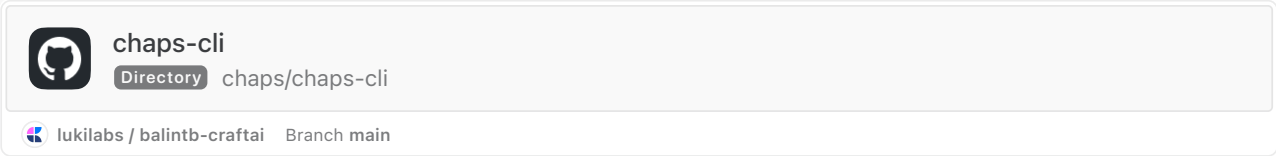
Did some thinking around SDK from the devs perspective. Not comprehensive, but thoughts here



Based on this we'll need some form of CLI tool similar to the likes of `wrangler`, `flyctl`, etc. And further based on this thinking, prototyped the chaps cli tool which would have these base commands:

```
chaps init hello-agent
chaps lint agent.yaml
chaps scaffold # add mcp
chaps push agent.yaml
```

In my thinking this form of interaction with chaps would be a good source of trust with the developer community. A cli tool is one of the first things I install when starting out with a new platform.



The tool is forward-looking - for example it uses `keytar` for auth (unlike almost all tools that use a simple file-based token auth option), browser localhost token capture. If we want to create a new product that resonates well with developers let's create world-class tooling for it as well.

Admin

Agreed with Levente we'll start using a shared GDrive folder for chaps infra & AI invoices - they're being paid with my company card 🙂

Next steps

- Mastra Merge (to) Main (on) Monday
- Implementation of Agent SDK by fusing together Sipi's thoughts and mine. This will be the theme of next week.

Levy Melnikov

2025. Jul 4., Fri

Stripe

I've implemented another Stripe prototype where balance is managed by Chaps.

- User's LLM credit balance is tracked & managed by us internally. There are two tables. By combining them together we get user's balance.
  - Usage - tracks LLM usage prices
  - Balance - tracks ad-hoc overage payments and beginning of the billing cycle credit grants
- Stripe is used for:
  - Subscription \ payment method management
  - Charging for overages as a one time payment(with a stored payment method aka saved card)

This way we have much more control and are much more flexible. The cons are:

- Saved card might decline the payment(e.g it's expired). We'll need a flow asking the user to check on their payment method.
- More data to store and manage: usage + payments + credits
- More logic: beginning of the billing cycle credit grants + periodically charging for overages
- More fragile operations: Charging the user, granting credits are fragile operations that need to be airtight: :"exactly once" semantics with idempotent keys.

2025. Jul 7., Mon

Billing - Balance management, use Stripe or build our own?

I want to go over two prototypes a bit more and prepare some learning \ questions on which do we want to go with.

Peter Sipos

Theme of the week: agent configuration, agent SDK, new agent features (chat menu, communication style, mcp api exploration)

[Update since Friday] PR before Mastra merging: bring back tool call user messages

Tool call user messages with native tool invocation
#76 · Opened by Schipy

Merged

↑ Daily Notes - 2025 July

Jul 4., Fri

Chaps
Levy Melnikov · Stripe · I've built a minimal flow with Stripe. A subscription has 2 prices: · One is a flat price for the s...

↑ Jul 4., Fri

Chaps

Levy Melnikov

Stripe

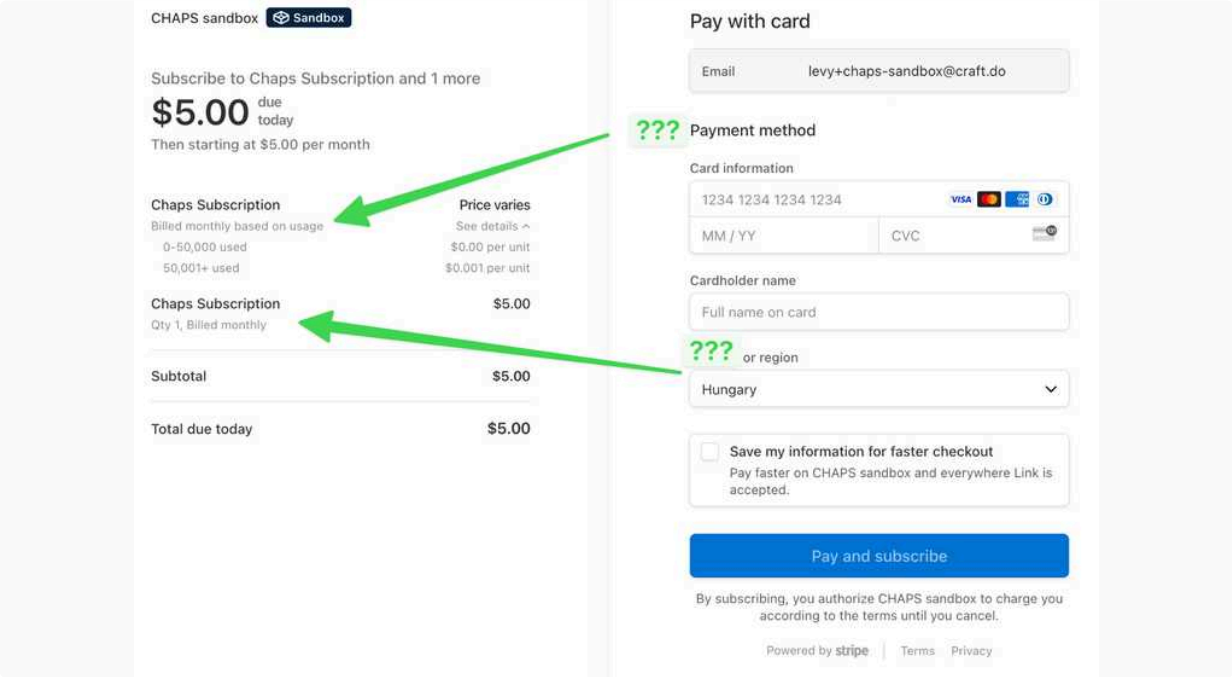
I've built a minimal flow with Stripe. A subscription has 2 prices:

- One is a flat price for the subscription itself: $5
- Another is a metered price with first 5 "chap dollars" free and then 1 "chap dollar = $1"

A background process reports all LLM usage to the meter. At the end of month user will pay $5 for the next month subscription and all overages that happened.

This is a very simple setup where user's credit balance is managed by Stripe. Some disadvantages of this setup:

- Although Stripe recommends this setup it produces a very puzzling checkout page where the same product is show twice(once for the flat fee and another for the metered usage)



- We report usage in units and units are whole numbers. Since LLM usage is usually very cheap(fractions of a cent) we have to assign a very small value to a unit. For the proto I set 1 unit = $0.001 , which also complicates confusing checkout page and invoices.

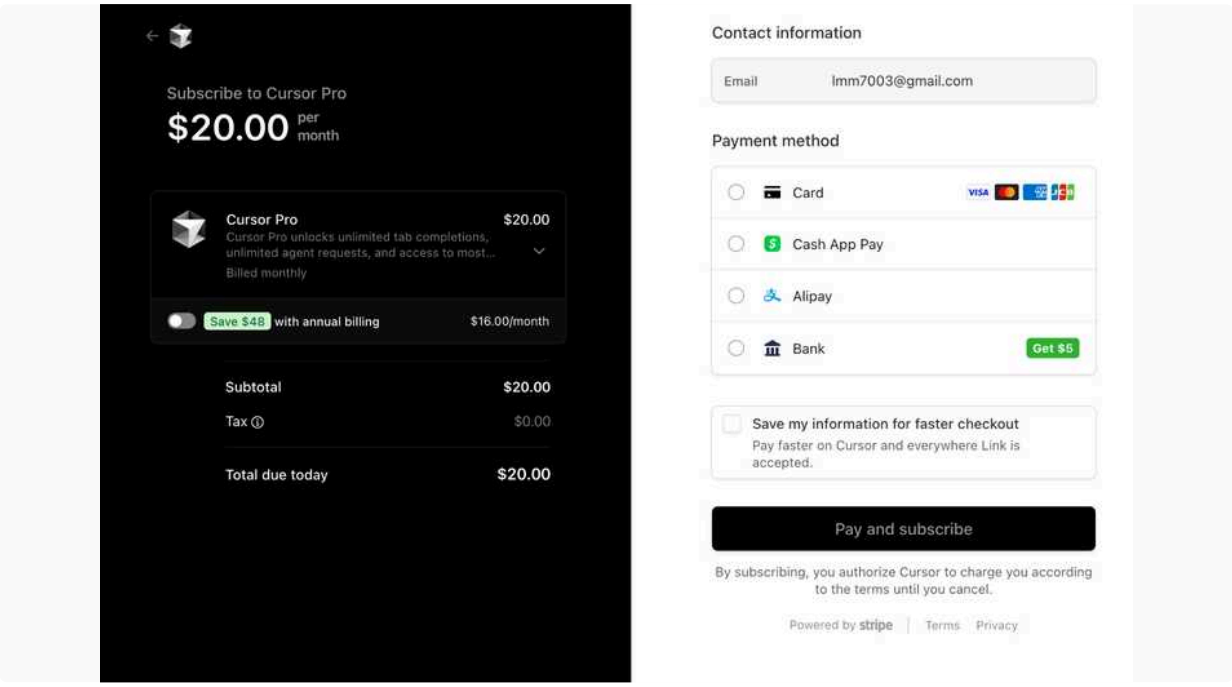
Description	Qty	Unit price	Amount
Jul 3 - Jul 3, 2025			
Chaps Subscription	1,536		
Unit price	1,536	$0.00	$0.00 ···
Flat fee	0		$0.00 ···
		Subtotal	$0.00
		Tax	-
		Total	$0.00
		Amount paid	$0.00
		Amount remaining	$0.00

- "Source of truth" about user's credit balance is in Stripe. To show their current balance we'll need to fetch an invoice draft and get the numbers from there.
  - This means that it would be impossible \ difficult to add any other payment method other than Stripe
  - Our usage stats and Stripe's might temporarily diverge since it takes some time for the meter usage to be accounted for in a draft invoice in Stripe.

Balance Management

Now I want to create a prototype where:

- User's LLM credit balance is tracked & managed by us internally
- Stripe is used for:
  - Subscription \ payment method management
  - Charging for overages as a one time payment(with a stored payment method aka saved card)
- ▼ ☝ This seems to be the way that Cursor handles it, as I see now mention of metered usage in their Stripe checkout page.



Peter Sipos

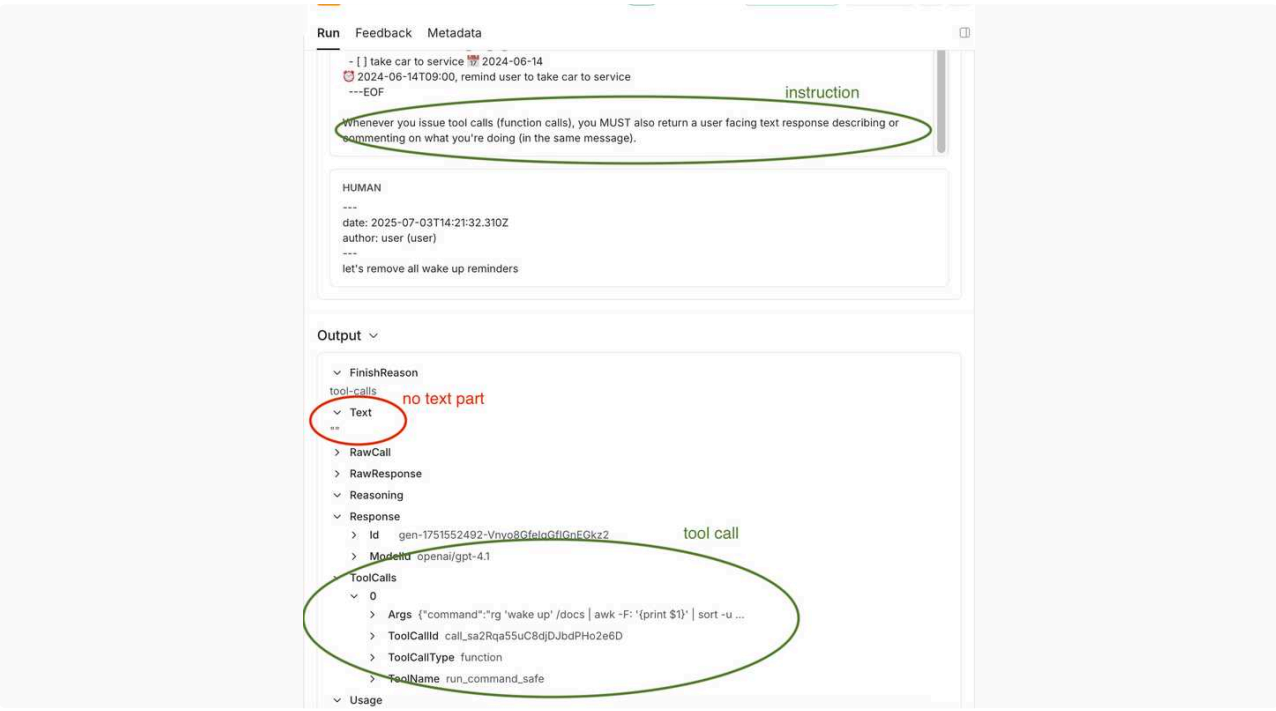
Mastra testing and fixes, eg

- Simplified model parameter loading from agent config (introduced a new `model` prop, which is the ai-sdk constructor in a strigified form, eg `openai(gpt-4.1)`, or `openrouter(openai/gpt-4.1)` This spared us backward compa parsing the old Orchestra names.
- Lot of code path unification where Mastra agents were created and executed to make sure that we have the same tool injection, context enrichment, progress callbacks etc everywhere
- Fixed some issues in ported secret loading
- also various smaller prompt fixes in agent.yaml (did 4.1 get dumber? 🤔, I had to clarify a couple of instructions regarding frontmatter and setup links etc, which went more smoothly before)
- also found & fixed a small glitch on magic setup link rendering

> The most interesting/problematic issue that I could not solve yet, is that with the new Mastra/ai-sdk "native" tool calling mechanism, we lost user facing tool call summaries, and could not bring them back yet. Needs a few more hours of investigation, and there's a plan B (see below)

Details:

- in Orhestra tool calling was implemented with custom instructions that explained a custom return json format, that explained the model to return a tool call summary as well with tool call jsons (if the appropriate Orchesra flag was on).
- in the provider & ai-sdk native tool_call messages, there's no such field, no place to put a user facing message
- the way to return a userfacing message on eg OpenAI api is to return multipart content, with a text message part and a tool_call message part. This is in theory translated to a `text` and a `toolCalls` fields in the ai-sdk/Mastra generation step finish response.
- This does not happen, even though I checked many pieces of the chain and it should work (the model supports it, openrouter supports it, ai-sdk and Mastra supports it etc). I also have a system prompt injected that tells the model to return a text part as well, but does not arrive so far.



As next step I want to eg play around in OpenAI Playground to see how this should work.

Plan B: we can augment every single tool call signature (automatically) with a mandatory `userSummary: string` argument, that the LLM has to fill to call the tool, which we forward as progress message.

2025. Jul 4., Fri

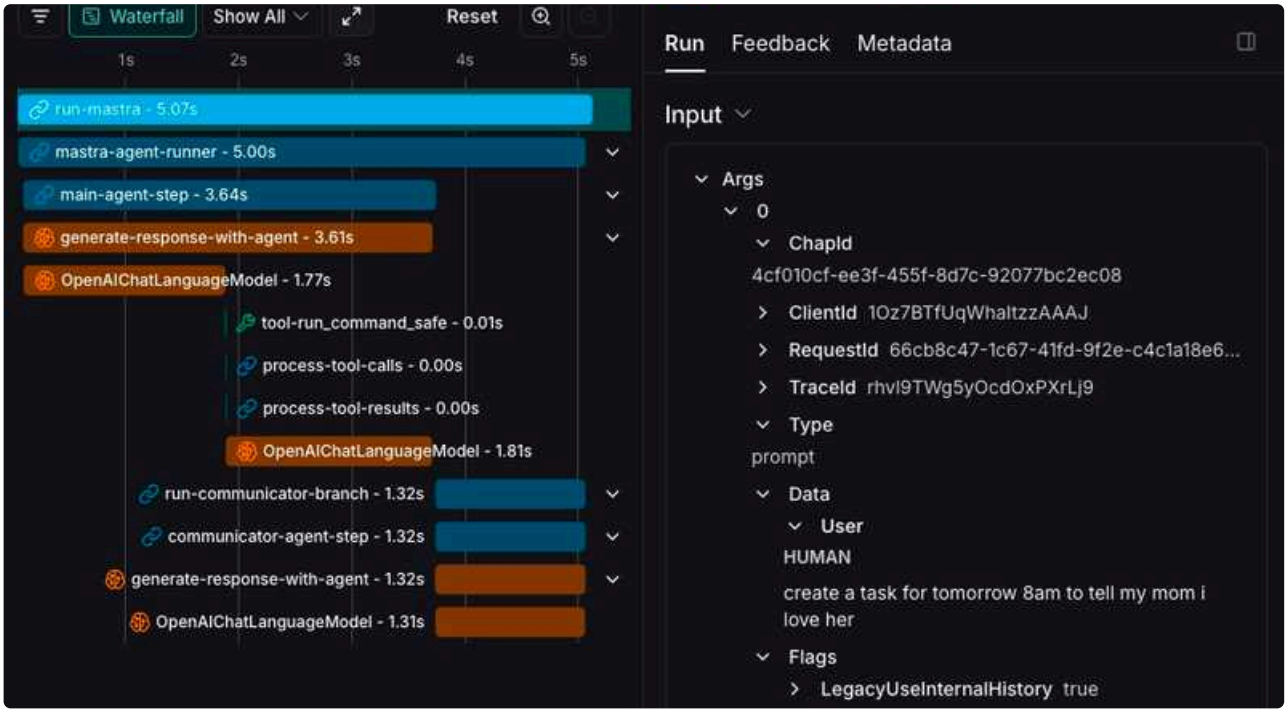
ooo (Friday), running errands like car service, so will have quite sporadic Slack checkins

Balint Bartfai

Mastra

Lots of thinking with Sipi around specifics of the Orchestra → Mastra port.

Spent most of he day on getting detailed tracing to work with Mastra and exporting it to LangSmith. I managed to get it to work - we're not only breaking down the user request by LLM call, but also by Mastra workflow branch. In the screenshot below, you can distinguish between Mastra agents, workflows, steps, branches, tools; and the LLM calls. Now, we're only using a conditional branch to see if we need a communicator agent or not, but we're already planning to add concurrent / parallel classification via a small & fast model - it'll be very useful in that scenario.



What we don't have yet is the cost, but I believe Levy's calculations will be much more precise anyway.

Interesting to see that **input / output token ratio is ~12 : 1** (4.16k / 350), of course only on the limited testing I've done today.

Evals

On branch:

Github
Directory not found
https://github.com/lukilabs/chaplet/tree/mastra/evals

Mastra has built-in evals which can be viewed by default with the Mastra dashboard, if we run Mastra server. We won't be running Mastra server, so I was working on exporting these evals to LangSmith.

I've added some simple metrics to our agents, which are evaluated by default on **every run** in dev, and can be **sampled** in prod. Evals run after the agent has responded. Current metrics:

Important for "main" agent:

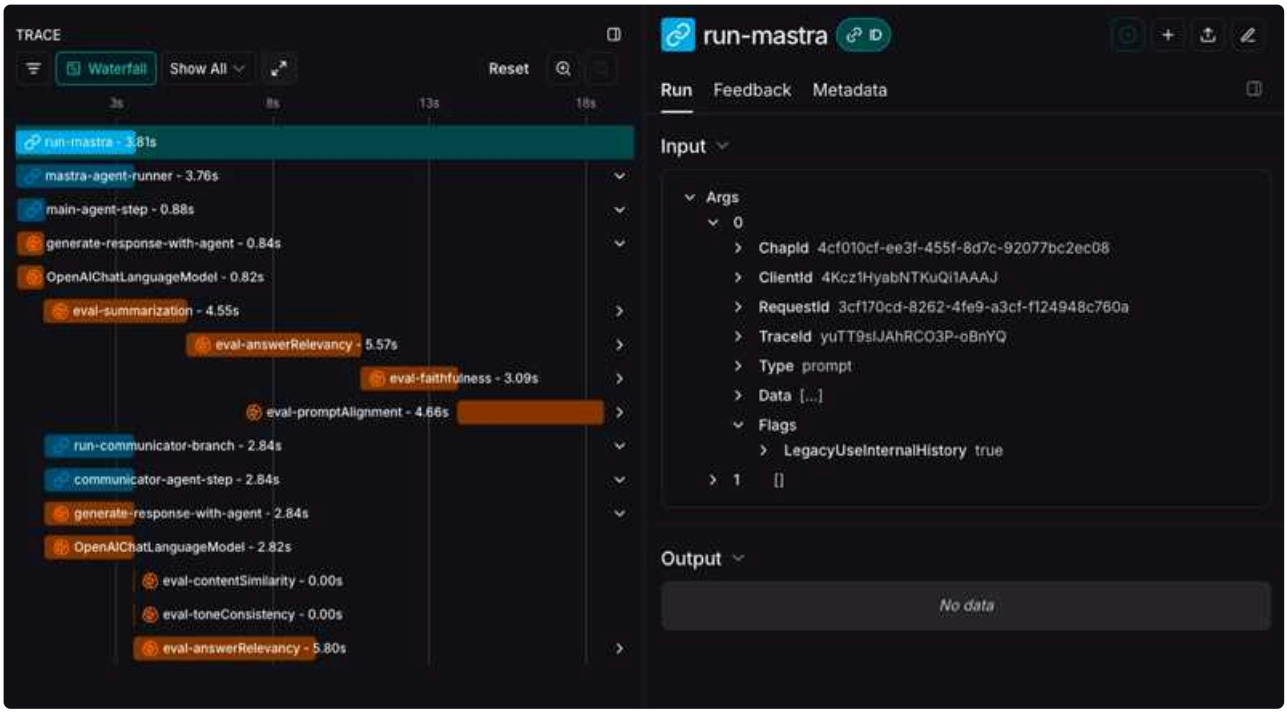
- `AnswerRelevancy` - how directly, completely response addresses actual prompt.
- `Faithfulness` - the "*hallucination metric*". Response contains only information that can be verified from context.
- `Summarization` - how well response captures and condenses key information from input. This is to ensure agent doesn't lose important details when processing more complex queries/context.
- `PromptAlignment` - how well response follows given instructions.

Communicator agent:

- `ContentSimilarity` - how much original content is preserved when reformatting & summarizing to ensure communicator doesn't drop information when presenting result to user.
- `ToneConsistency` - whether tone, style remains consistent throughout response.

We'll see which of these are more important than others based on testing and real-world usage.

This way, the traces look a little different:



Steps starting with `generate-` run the chap itself. Steps starting with `eval-` only run the evals async afterwards. Evals don't effect the response speed.

In this example, `whats the capital of hungary?` took 3.81sec: 0.88sec for the answer, then 2.84sec for the communicator. Evals are submitted 5-6 seconds later, some sequentially, pushing the trace to 18sec.

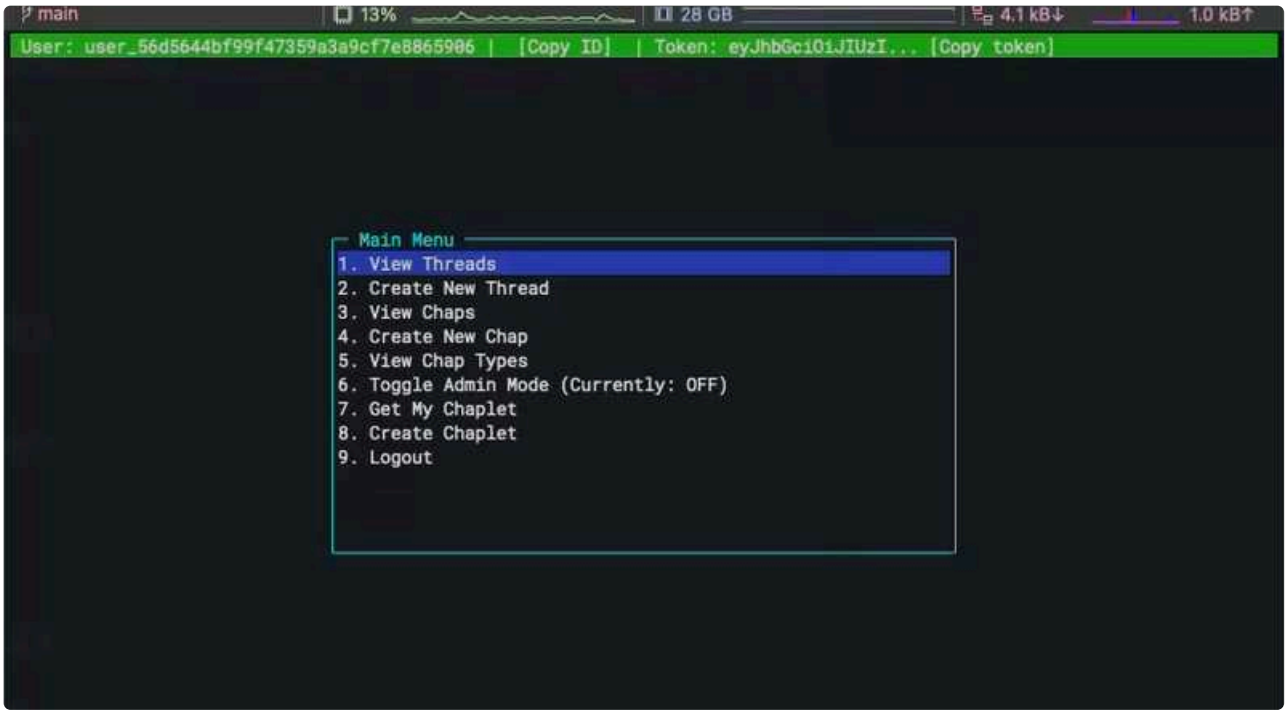
These metrics are visualized on a dashboard:

As a next step, I'm starting to think about custom metrics we could use with LLM-as-a-judge, and potentially quantitative ones too. It would be great to be able to define this per agent - a simple reference in config to a Typescript module which receives context and does the calculation.

TUI

Sipi's wishlist improvements including

- more clickable clipboard IDs
- socketio events



Next steps

Mastra is in a good spot now, we're planning to merge it (switch over) Monday. I'm going to be simplifying the LLM settings, as now we map them all over the place. We'll use OpenRouter, so I'll map them to that only. Use a whitelist.

Agent SDK "config" planning. How do we get from a simple config file (JSON, YAML) to the more detailed chap configuration. An interesting aspect I'd like to tackle:

Let's say we have a property in the config, which a seasoned developer can set up:

```
{
  ...,
  tools: [
    tool1: {...},
    tool2: {...},
    ...
  ]
}
```

How could a junior developer replace this in the "external" chap config with:

```
{
  ...,
  tools: "ability to store oauth tokens",
  ...
}
```

This would cater to novices who would like to create agents.

↑ Jul 4., Fri · Chaps

State of plans & current thinking regarding chap configuration

cc Balint Orosz Balint Bartfai Levy Melnikov

Manifest approach

This is something we discussed with `BalintO` on Wednesday.

Chaps will have a manifest-like config that describes capabilities. In the Beta, this effectively defines the 3rd party chap completely. Not everything will be exposed in this manifest, so our own chaps will have more logic than what 3rd party devs can create for now.

- there will be no config UI in Beta

- rather the developer user will be able author the json with his/her json editor of choice (we give schema) and then provide/upload this manifest, which creates a new chap type, and can then instantiate the chap. Not sure what is the staging area for this chap type, as submit to actual registry should happen only once the dev has tested and is satisfied with the chap.
- in the Beta, for tweaks / dev iteration he will need to reupload the manifest (though we could offer dynamic update fairly easily in an already running chap, but we plan to cut this from scope)

I started creating this manifest a few days ago, which is called `composition.json` The json itself is more of a placeholder yet, but I PoC-ed the pipeline to render low level configs from high level capabilities (see this update).

Chap definition

A potential list of features in Beta that we have either already discussed together and some I'm dropping in now after recent day's thinking.

MCP server

(remote and npx, with simple params)

for now only a single one for simplicity, multiple MCPs brings in a lot of complexity (eg lot of other features are scoped to a particular MCP)

default responding LLM model

likely we will only allow openrouter names (or translate everything to openrouter)

maybe also temperature

define overall chap goal and purpose

set chap communication style (tone of voice)

default, secretary, butler, u-da-boss, talk-like-a-pirate etc, custom, this is experimental, but I see it as an interesting factor that could spice up the whole thing, like how reactions made Slack very slick in the beginning.

this is likely more attached to the user, no so much the chap, so may be a base chap feature (to be able to switch and/or define custom styles)

define additional MCP tool knowledge

custom free prompt, that dev can give to clarify/enhance MCP tool usage of the LLM (filling in documentation gaps of MCP server basically), in current language this is the initial content of _tool_learnings.md

define MCP data caching strategy

custom prompt that gives hints what to cache from the MCP api (eg for GCal, cache the calendars, in Linear cache the Teams etc), We run the LLM with this prompt included to generate an initial memory file that's added to context. Later can rerun on some schedule.

enable/disable web search

maybe set limits, like number of queries per request

agent bookkeeping/memory with a template (may be multiple)

defines a *thing* that the agent is instructed to bookkeep, specified by an instruction prompt, and an optional markdown template (the agent will create a file/files to maintain this THING).

this similar to Mastra's working memory .md template / also a kindof generalization of our `todo_chap` (or yet in other terms a simple version of our earlier template chap concept)

define configuration variants

define named config *overlays* - partial `composition.json` definitions with a unique name within the chap, which essentially define possible behavior changes of the Chap. Sounds cryptic, but could be a fundamental building block and engine behind a lot of capabilities, see below what they're good for.

define chat menu widget

this is a fixed format configuration file, that can both be manually specified (see fixed options below), or generated by LLM with developer guided prompt

fixed behavior switch menu: a fixed list of configuration variants (see variants above) to enforce in the thread with user facing labels. This is very powerful, that can be used for example to implement:

- LLM model selector: eg a `fast` and a `smart` mode, which behind trigger different LLM model config overlays
- Agent comm style selector (again simple config overlay with a different style)
- enable/disable web search button (config overlay on enable/disable web search)
- even an *account switcher* (by allowing same MCP server with different name as config overlay, we can maintain multiple auth tokens per MCP server, and switch which one to use in a particular thread). This needs some dynamic aspect to config variants ("add new account"), so not out of the box yet, but interesting prospect.
- etc

scope selector

custom prompt that tells to convert some scope-like property from the MCP api into a selectable menu, like project, team, workspace (activates a flow which uses LLM to generate the menu in the background based on MCP calls)

fixed quick prompt list

list of quick prompts to fire (eg in gcal chap, a one tap trigger to show daily schedule)

adaptive quick prompt list

custom prompt that guide how to extract frequently used prompts of the user

some other properties, like how often prompts should fade, how many to keep in quick menu etc (activates a flow which uses LLM to generate the quick actions based on conversation history)

define a chap setting

define a user facing setting that controls a named config variant. Can be used to achieve similar features like with menu, but exposed via a setting api (which is maybe behind a dots button on Chaps UI, or you can simply ask the chap to change the behavior itself, like from now on use a different named comm style)

Seems like a long list, but some of it is very simple prompts or mapping (like chap goals, model selection, additional MCP tool knowledge), and I think almost none of them are very complex, and once we/I can really get to it, we should be able to have many features from this menu, even for Beta.

Low level config (agent and lifecycle yamls)

The high level `composition.json`, so the 3rd party developer defined chap capabilites get rendered to our internal, lego-like current config yaml system, which is quite low level, but very flexible. I would not do fundamental changes to this low level config, at least not in Beta timeframe, to keep the scope feasible. On longer term we can if course revisit the low level config structure as well. However even in Beta we may need to add certain low level features, like scheduling, message history processing, LLM tool to create the menu etc.

↑ Daily Notes - 2025 July

Jul 3., Thu

Chaps Team

Levy Melnikov · Wed, Jul 2 · Usage Tracking · We now have a central table with all of the LLM usage. It's quite flexibl...

↑ Jul 3., Thu

Chaps Team

Levy Melnikov

2025. Jul 2., Wed

Usage Tracking

We now have a central table with all of the LLM usage. It's quite flexible for extending it with future use cases like paid tool calls(web search and so on). It also contains price information for each request. Currently rates for LLMs are taken from OpenRouter pricing.

There are also endpoints for getting usage \ costs per user, chap and traceId. TraceId is an interesting one: we can analyze how expensive each request to chaps is, for example **asking todo chap to set a simple reminder is 3 cents**. I'm not sure if these APIs will be used on clients - depending on how we integrate with Stripe it might be the best to get current usage \ credit information from Stripe and limit our backend to just correctly report usage to Stripe.

Next step would be to integrate it with Stripe. And here I have some questions:

- How to get access to our Stripe org?
- What is actually our pricing model? I will start working on a doc with a bunch of questions around it so that we can have a productive discussion about it.

I'm not blocked yet as I want to prepare a doc with questions, but soon I will be blocked on this topic until we discuss & finalize the v0 of our pricing model.

2025. Jul 3., Thu

Stripe

Start exploring integration with Stripe. In case I'm blocked I will get back to testing new k8s infra under various exceptional \ stress conditions.

Peter Sipos

Worked on Mastra migration the whole day pratically (aside from a bot of thinking and discussion with BalintO about chap use cases and setup experience)

Mastra

- Went through the PR, discussed tasks with BalintB
- Migrated agent config loading, merging, reimplemented config interpolation
- Migrated secrets loading
- Migrated context enrichment (context step parsing and evaluation)
- Migrated MCP server file config injection
- Added a lot of typings
- Bunch of fixes, test run again, fix again cycle

I feel we're quite close to e2e runs (few hours)

Next

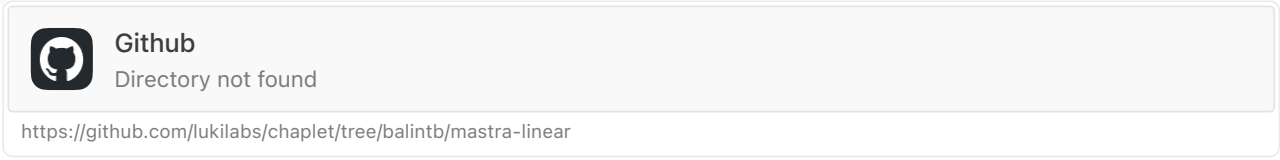
Would be great to reach parity with Mastra this week

If I have any time on the side, then we have discussed next agent features with BalintO already that I can start on

Balint Bartfai

Mastra

Worked on Orchestra → Mastra porting with Sipi today.



What has been done:

Streaming vs. Generated

Rewrote the Mastra integration to not use streaming for now. Replies from the model arrive in one, which matches the previous behaviour.

CLI tool

After streaming was redone, I got CLI tool working again. Some events were slightly different in Mastra, until we rethink the Model → Gateway → Client events I've made it compatible, events are the same as with Orchestra. CLI tool is the primary tool we use to test chaps while under development.

Support Agent

Resurrected the support agent which helps with configs, etc. Some rewiring was needed with the configs to make it Mastra-compatible.

Communicator Agent

Chaps has a two-step comms process: the execution agent will output an internal answer and make changes to the filesystem. The diff of these changes along with the internal answer is passed to a communicator agent to reply to the user with what's been done.

In Orchestra, we hardcoded this process, running the main agent from config, then passing enriched output to another agent. In Mastra, I used their workflow concept to control this flow with conditional branching. In some cases, the communicator agent is an unnecessary step ("hi!") - we can control the next execution with a conditional branch. The condition can be as simple as a flag (has comms yes/no), or depend on the output of the previous agent - as well as giving us the flexibility to run, say, a tiny fast model in parallel to the main agent, and decide whether to move to the workflow's next step based on main and tiny agent's output. Any changes to this process - ie. pre-classifying requests, post-processing (delivery method for example), parallel processing, etc, can be defined in this workflow, without hardcoding business logic. I'm sure we can do more with it, but for now this is a good start with a simple Mastra workflow.

TraceID propogation

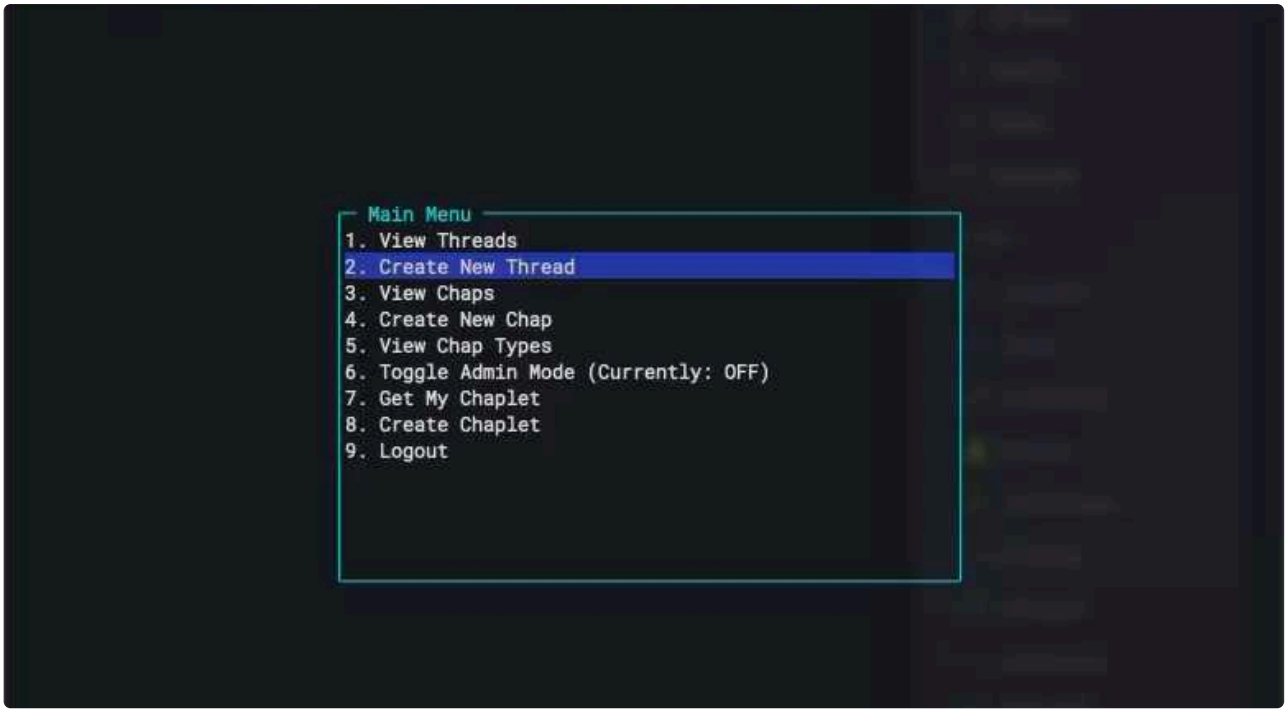
...and other small tweaks - typing, some Mastra-specific edge-cases, etc.

Simple chaps run on Mastra without problems now. Linear and GCal are still being tested & worked on.

Sipi has a definitive list of what is outstanding.

Side project: Client TUI

When building infra I realised how much time can be spent in Postman making API calls, copying IDs - even if scripted. So on the side of the Mastra work, I built - with lots of Claude generated code - a client TUI for chaps which works exactly like the iOS app + dev features. It speeds up testing E2E flows - makes all API calls under the hood for auth, etc., connects with sockets, has a fully functional chat interface, has token generation for users, admin mode, and copies IDs & tokens to the clipboard.



lukilabs / chaps-tui
github.com/lukilabs/chaps-tui
lukilabs TypeScript Stars 0

Next steps

Updated delivery plan.

- Mastra porting. Should finish tomorrow, but latest Friday
- Revisit Agent SDK planning
  - configs definition - based on Mastra work, where we will be changing some configs to better align with TS & the framework

↑ Daily Notes - 2025 July

Jul 2., Wed

↑ Jul 2., Wed

Chaps Team

Peter Sipos

Introduce high level agent configuration modules
#74 · Opened by Schipy

Merged

introduce high level agent configuration modules (/capability)

a chap can now have a `composition.json`, which describes a set of parametrized agent capabilities, for example having access to an MCP Server (in a similar high level way as in a claude config)

```
{
    "id": "mcpServer",
    "version": "0.1.0",
    "params": {
        "name": "linear",
        "server": {
            "type": "remoteMcp",
            "url": "https://mcp.linear.app/sse"
        },
        "auth": {
            "type": "dynamicOauth",
            "resourceUrl": "https://mcp.linear.app/sse"
        }
    }
}
```

when this capability module is rendered (currently at install time), it produces

`agent-linear-mcp.yaml`

```
tools:
  linear_mcp:
    type: mcp-sse
    server_name: linear
    sse_url: https://mcp.linear.app/sse
    sse_headers:
      Authorization: Bearer ${secrets.oauth.linear.access_token}
setups:
  default:
    agents:
      main_agent:
        tools:
          linear_mcp: ${tools.linear_mcp}
```

and `lifecycle-linear-auth.yaml`

```
user_steps:
  linear-connection-setup:
    type: oauth.authorize
    provider_name: linear
    client_config:
      resource: https://mcp.linear.app/sse
pre_agent_execute:
  linear-connection-refresh:
    type: oauth.refresh
    provider_name: linear
user_step_change:
  linear-apikey-setup-completed:
    from: incomplete
    to: completed
    user_step_id: linear-connection-setup
    action:
      - type: run.prompt.chat
        prompt: User successfully connected to (authorized with) linear. If not other
          essential setup is left, the Chap should now be fully functional.
        threadTarget:
          type: last_active
```

which implements the mcp tool and the oauth lifecycle in the chaps config system

these rendered configs are merged into the final config just like all previous sources of configs (base, chap, dynamic) etc

In this PR the Linear chap has been updated to use the new `mcpServer` module.

Other notable aspects:

- config/capability modules have a versioned manifest, define zod types for runtime type validation. They will also contain a json schema version should editors need them, plus UI label information for building settings interfaces for them.
- they're also extensible regarding implementation types, for now the supported method is `nunjucks` templating (which is the TS port of the well known Python `jinja2`)
- to make this work, I had to make `lifecycle.yaml` files mergeable, just like how agent configs were mergeable (eg using dicts instead of lists on top level) - this is something I wanted to do anyway

there's still several pieces to add and many polishing to do, but felt it's getting big, so going for a stable checkpoint here.

Next

I'm thinking about working towards "e2e" experience rather than breadth, meaning work on the pipeline and less about including more capabilities. Rough goal:

- user can create "blank" chap ("Create your own")
- pulls up a config UI, where he can set MCP server(s), main prompts of the chaps, maybe a few checkboxes.

- the chap works according to the configuration

Later

- how is this dynamically configured chap promoted to a Chap type? How is it shared?

However it's also an interesting angle to map out how the top level capabilities and config structure would look like for users (expecting to also spend time with this on the sideline):

Example capability - "Chap MCP memory":

- fixed data specified by user (custom chap, not published chap type)
- automatic MCP API exploration by LLM
- guided (prompted) API exploration by LLM
- programmed (templated) MCP calls to memory mapping

Example capability - "Chat menu widget":

- fixed dropdown (custom chap, not published chap type)
- guided (prompted) scope dropdown menu generation by LLM
- programmed (templated) MCP calls to dropdown menu
- also same options for not a scope selector, but quick prompt list as widget

Levy Melnikov

2025. Jul 1., Tue

Big Infra Changes Finalization

I've finalized and deployed to prod 4 big changes:

- `platform.chaps.app` → Digital Ocean App
- `api.chaps.app` → K8S on Digital Ocean
- `auth.chaps.app` → Use neon Postgres for storage
- Chaplet state → R2 bucket

⚠️ This erased all state. You will have to create new users and a new chaplet with empty will be issued for you.

Cluster right-sizing

I've looked into right-sizing chaplets and out cluster, learnings:

- Better to go for bigger nodes → nodes reserve some RAM for the system. So if a node has 8GB, we can only allocate 6GB for chaplets. Ratio of system / all RAM is better in bigger nodes.
- We should go for ~100 chaplets on a node → this restriction comes from K8S, it can't host more than 108 containers on a node.
- We are limiting RAM at 500 Mib → based on metrics from flyio this is 1.5 times bigger than what chaplets consume
- We are limiting CPU at 1vCPU → arbitrary number. Actually there's less cpu on a node, so if all of them go active at the same time they will start to compete for CPU
- ⚠️ With current setup we should be able to host 50 chaplets.
- ⚠️ By requesting access to bigger nodes from Digital Ocean we can host ~10.000+ chaplets
- ⚠️ With current resource limits, price per chaplet is ~5$ \ month

2025. Jul 2., Wed

Usage tracking & billing

Start (re)implementing usage tacking & billing. Roadmap would be:

Properly store all usage: this time I'd go for storing on per-invocation basis not just aggregated data. This will help us later with stripe integration. Also I'd like to make the table a bit more future proof and include use cases like web search usage(which is billed separately from llm).

Setup pricing info: we have to know what we are paying for each LLM request and what a user will be billed for each request. This will require to setup a table that would say 1 token of model X cost us $Y and is billed at $Z.

Prepare user facing APIs: to get usage\cost per account, per chap, per message.

Ensure proper tracking: we will need to update the logic on the chaplet to use this new API. Also there's this server web search that is not tracked at all yet. Also with Mastra tracking might need to be re-implemented.

Look into Stripe: Start preparing a PoC type of flow where you can pay for your usage.

Balint Bartfai

ℹ️ I will be working from home tomorrow, probably unavailable from 9:00-9:30 due to maintenance in the apartment

Orchestra → Mastra

Original plan was to try this chap-by-chap. However, our implementation is so embedded into orchestra, it made more sense to try and replace the orchestra service starting from the lowest layer. So I worked on moving the low level execution layer from Orchestra/Python to Mastra/TS.

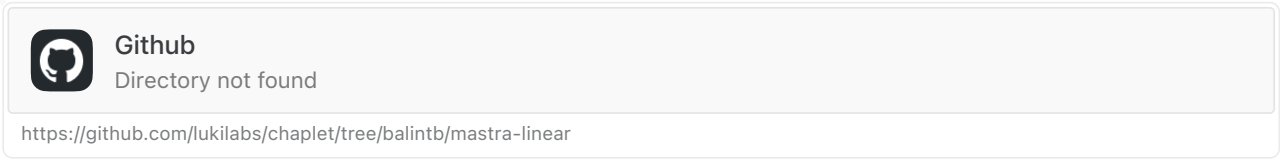
By the end of the day I had a chap running (todo chap). This will need much much more testing to verify that the changes to tasks as seen previously actually work with the new framework and tool calls. I can see tools being called, haven't verified the full behaviour is the same. To make it work multiple components needed to be move to Typescript

- config handling. Having talked to Sipi I'm pretty sure it's not complete for more complex chaps (→ config interpolation)
- Tools and tool configs. Python logic to TS. Some tools can't be ported in a straightforward way - luckily we don't use them. Human input is an example, Mastra treats situations like this differently and solves them with workflows
- Execution itself. All agents built and run in Mastra

Config & templating is quite tied in to Orchestra & Python, which will need some more rewriting if we want to do this in typescript.

No complex oauth chaps yet - I'm fairly certain more complex chaps won't work out-of-the-box with this setup. But for those, I'll need help, as this is unfamiliar territory.

Branch can be found here (Peter Sipos 👇)



(comparison to main here)

Performance

In terms of performance - and this in no way is a sterile laboratory test - the rough figures for different prompts (todo chap) looked like this:

- Model: `openai/gpt-4.1` via OpenRouter
- E2E time: prompt submission → reply stream end. Response stream starts almost immediately with this model, and is streamed word-by-word.

Prompt: `hello`

Response: `Hello! How can I assist you today?`

E2E: 1,89sec

Prompt: `what would be a good use of my time for the rest of today and the whole of tomorrow?? i need to walk my dog for 10 minutes, and need breaks of at least 15min betweeen activities`

▼ Response

```
To make the most of your time today and tomorrow, here's a balanced schedule that includes productivity, relaxation, and leisure:

### Today
1. **Evening Walk (10 min):** Take your dog for a walk. Enjoy the fresh air and a bit of exercise.
2. **Dinner (30 min):** Prepare and enjoy a healthy meal.
3. **Relaxation (1 hour):** Watch a movie, read a book, or listen to music.
4. **Planning (30 min):** Plan your tasks for tomorrow. Make a to-do list.
5. **Wind Down (1 hour):** Unwind with a calming activity like meditation or a warm bath.

### Tomorrow
1. **Morning Routine (1 hour):** Start your day with a healthy breakfast and some light stretching or yoga.
2. **Work/Study Session (2 hours):** Focus on a project or learning something new.
3. **Break (15 min):** Take a short break to relax.
4. **Creative Activity (1 hour):** Engage in a hobby like drawing, writing, or playing an instrument.
5. **Lunch (1 hour):** Enjoy a nutritious meal and some downtime.
6. **Outdoor Activity (1 hour):** Go for a walk, bike ride, or any outdoor activity you enjoy.
7. **Break (15 min):** Rest and recharge.
8. **Productive Task (2 hours):** Work on a task or project that requires focus.
9. **Break (15 min):** Take a short break to relax.
10. **Evening Walk (10 min):** Take your dog for another walk.
11. **Dinner (30 min):** Prepare and enjoy your evening meal.
12. **Relaxation (1 hour):** Unwind with a favorite activity.
13. **Reflection (30 min):** Reflect on your day and plan for the next.

Feel free to adjust the schedule to fit your personal preferences and commitments. Enjoy your time!
```

E2E: 13sec

Prompt: add a task for tomorrow afternoon to make my favourite lemon pie. Find a good recipe for it!

▼ Response

```
It seems there was an issue with saving the task and recipe. Let me try a different approach to ensure you have everything you need for making your favorite lemon pie tomorrow.It seems there is a persistent issue with saving the task and recipe. Let me provide you with the recipe here so you can have it ready for tomorrow:

### Lemon Pie Recipe

#### Ingredients:
- 1 cup sugar
- 3 tablespoons cornstarch
- 1/4 teaspoon salt
- 1 1/2 cups water
- 2 lemons, juiced and zested
- 2 tablespoons butter
- 4 egg yolks, beaten
- 1 (9 inch) pie crust, baked

#### Instructions:
1. In a saucepan, combine sugar, cornstarch, and salt. Stir in water, lemon juice, and lemon zest. Cook over medium-high heat, stirring frequently, until mixture comes to a boil. Stir in butter.
2. Place egg yolks in a small bowl and gradually whisk in 1/2 cup of hot sugar mixture. Whisk egg yolk mixture back into remaining sugar mixture. Bring to a boil and continue to cook while stirring constantly until thick.
3. Pour filling into baked pastry shell.
4. Preheat oven to 350 degrees F (175 degrees C).
5. Bake in preheated oven for 10 minutes, or until meringue is golden brown.
6. Cool on a wire rack.

Enjoy your delicious lemon pie!

Please make sure to set a reminder for tomorrow afternoon to make your pie. If you need further assistance, feel free to ask!
```

E2E: 23sec

**here it seems like a tool call failed*

These numbers don't mean much without the Orchestra comparison, but, subjectively, it *feels* faster in terms of latency. CPU load avg, after load dropping down post apt installs, was `0.7 / 0.12 / 0.2` for a longer prompt. This is less than the Python version spiking to `2.06 / (?) /(?)` in a previous run. I plan to do a more comprehensive experiment once other chaps are working as well, as close as possible to the two setups.

Tomorrow

- Evaluate with Sipi if this switch is doable based on today's work
- → find solutions to config loading
- → complex chaps (OAuth)
- Update delivery plan based on Mon/Tue + Wed estimated progress on infra & agents

↑ Daily Notes - 2025 July

Jul 1., Tue

Chaps Team

Peter Sipos · workshop in the morning, in the afternoon also a sync with BalintB about Mastra next steps, and confi...

↑ Jul 1., Tue

Chaps Team

Peter Sipos

workshop in the morning, in the afternoon also a sync with BalintB about Mastra next steps, and configuration topics

`merged` GCal chap improvements

chore(build): create a self contained bundle, and drop excess googleapis dependencies

update gcal server plus enrich user calendars into context

updated GCal MCP server + plus context enrichment with user's calendars

`needs new Chap!`

- created a proper private fork for the GCal MCP we're using: https://github.com/lukilabs/google-calendar-mcp
- merged in upstream changes since last month, which includes improved timezone handling, and a bunch of refactors
- I specifically needed this, as upstream also improved list-calendars tool, which I wanted to use context enrichment
- added the this context enrichment, ended up as an ugly template, as the mcp server outputs the calendars as free text (for "convenience", but for us actually a hurdle, as we would be better off with structured data)

  Example agent enrichment output:

```
The user's calendars are:
  Holidays in Hungary (en.hungarian#holiday@group.v.calendar.google.com)
    Timezone: Europe/Budapest
    Access Role: reader
  Gerg\u0151 / Csal\u00e1di
(3veiurd26knuc8ub78ht6t1auk@group.calendar.google.com)
    Timezone: Europe/Budapest
    Access Role: writer
  P\u00e9ter Sipos (schipy@gmail.com)
    Timezone: Europe/Budapest
    Access Role: writer
  Peter Sipos Work (PRIMARY) (schipy@craft.do)
    Timezone: Europe/Budapest
    Access Role: owner

```

Solved size of GCal MCP server

pushed down size from **206Mb to 1.5Mb** ((full node_modules with many thousands of files to a single cjs file)

- changed its build to bundle dependencies, instead of requiring the `node_modules` to be present at runtime, for this to work needed to switch to using `commonjs` module to handle `require` calls bundled into the module (size moved down to a 11Mb bundle)
- narrowed import from `googleapis` to `@googleapis/calendar` (and googleapis-common), as a escalating effect needed to switch and use `OAuth2Client` from `googleapis-common` instead of `google-auth-library` (size moved down to 1.5Mb)

`experiment` Automatic context enrichment experiment

since we're aiming for non tech people (or at least non devs) to be able to create chaps, I started exploring automatic ways of enriching context. Instead of chap developer needing to define template steps in configs (needs actual developers, see above), we run an LLM on the MCP api to figure out what would be nice to cache as context enrichment. We talked about doing this later based on past conversations (extract useful context), but I think for Chaps to be successful/viral, things have to work fairly good *right away* after you spoin up a Chap (as first impression). so I believe context enrichment is important right from the beginning.

I mocked this up and had a test run with o3 on the Google calendar API, and had already promising results:

It generated the following example context for the main MCP agent (before any conversation is happening)

```
# == MCP bootstrap ==
userTimeZone: "Europe/Budapest"
bootTimestamp: "2025-06-30T09:35:12Z"
primaryCalendarId: "primary"
calendars:
  "primary": "primary"
  "Team Calendar": "c_team12345@group.calendar.google.com"
  "Personal Reminders": "c_reminders@group.calendar.google.com"
colorMap:
  "1": { name: "Lavender", bg: "#A4BDFC", fg: "#000000" }
  "2": { name: "Sage",     bg: "#B7B7B7", fg: "#000000" }
  # …
# ====================
```

- Detailed assessment of the generated context
  - bootTimestamp is current time, not needed, I didn't explain that main agent will be aware of time. If I did, fairly sure this would not have been added.
  - calendars: good result, access rights per calendar would have been a useful addition (like I add it in above procedural enrichment)
  - colorMap: not needed for us, but it could not have known, as the chat context was not explained in this prompt. I guess it assumes system has a visualization for events, for which colors are handy to have at hand. Which is actually could even be the case.

with some iteration on prompt (that generates this), this seems usable enough for me → this seems like a viable capability/module that we can experiment with.

Technically this is something we can **already** configure:

- define a new agent, describing the task (query and save useful context on an MCP api)
- on auth setup complete lifecycle hook, run a non-chat emitting prompt action (we have this already), which triggers this agent
- add context step for main agent to include this saved context in its context
- the only shortcoming is the lifecycle part (setup hook) is not modular yet, so this is config would have to be hardcoded into a chap, cannot toggle it on/off easily as we want behaviors to be

`explore` Onto high level Chap configuration

I was thinking a bunch about how to approach the chap configuration given the goals we discussed on the workshop. I think I have some handle on in now, and tomorrow will start prototyping (plus also sleep on it)

a rough view:

- we define *capability* modules / sub modules, which are coherent pieces of behavior with simple configuration APIs. A high level (parent) module is "access to an MCP server", a lower level module may be is "explore MCP server before use" (above mentioned auto-context enrichment).
- modules define their api as json schema, and may even include UI descriptors/assets to facilitate generating the preferences UI for themselves.
- the module implementations' job is to compile to the low level yaml configs we have now (and to extensions/enhancements of that). Simple modules just include yaml config snippets with some string templating, while complex modules can work even by TS code generating the YAML. Later the underlying low level config implementation can be changed without changing the high level module API.
- Modules can also vary regarding complexity/audience, take enrich-initial-context-with-cached-MCP-data problem:
  - the entry level module is an automatically included setup, where we run the generic LLM prompt to generate context after MCP server has been set up
  - medium level module offers the author to customize the prompt of the exploration LLM round, eg can explicitly give hints to LLM what to cache from the API
  - advanced level module for the same purpose allows author to define actual procedural steps to map MCP calls to context with templating, like how I do now in the chap configs.
- a chap's top level configuration (in terms of agents and lifecycle) is a set of modules with their params. It may also have a structure, eg have a slot for “menu1” module or “communication style” if we want
- the chap type can be defined either by a fixed chap config (above top level config), or a general chap type can be defined by the set of allowed modules (where the specific config is dynamic within that range), not yet clear which way we would rather lean (what is the actual "develop new chap" experience)

Levy Melnikov

2025. Jun 30., Mon

Platform APIs

In the morning I continued to test the new cluster and saw that platform is not working on sandbox. We decided to not fix it on flyio and instead move it to digital ocean and solve it there.

- Set up secrets, envs, domains for sandbox and prod
- Setup CI\CD and deleted old flyio CI\CD from the repo
- There was a syntax error in the code that prevent the app from starting → I solved it and also added building & linting a part of CI\CD and build process so that we never have this issue again.
- Checked out this commit. `chapType` and `chapTypeVersion` should not be deleted they are part of the chap registry - I reverted that and left only the deletion of `chaps` table which indeed should be removed as we track chaps in auth now.
  - Also after making a schema change we also need to generate migrations with `docker-compose exec api drizzle-kit generate` and commit it. After that apply it using `scripts/migrate.sh` on sandbox and pord (I fixed the script to use DO) cc Balint Bartfai

✅ I verified that new DO platform API works. I routed existing `platform.[chapsdev.com\chaps.app]` domains to DO. We can now remove it from flyio.

2025. Jul 1., Tue

Chaps on K8s

Continue with e2e testing. We recently did a lot of moves:

- ✅ `platform.chaps.app` → Digital Ocean App
- ⚒️ `api.chaps.app` → K8S on Digital Ocean
- ⚒️ `auth.chaps.app` → Use neon Postgres for storage
- ✅ Chaplet state → R2 bucket

I want to make sure that all of these work both in sandbox and prod and properly close these down, before proceeding with usage \ billing tracking.

Balint Bartfai

~4h meetings, planning

Sync on workstreams, planning

- Levy: migrating over sandbox cluster + systems to prod, then continuing onto better usage stats
- Sipi & me: Agent SDK, Mastra

Had a longer planning session with Sipi around Agent SDK. There's still a lot of open questions, but both of us have started to proto ways to solve the "configure my chap" problem from top-down (Sipi) and bottom-up (me).

Things we agreed on for the first iteration:

- custom chap creation will be config-driven - either host your config or upload it to create the chap template
- The simplest of custom chaps (my grandma) will be just a description in a property of a config file
- the lowest-level (a seasoned developer) will be some form of custom-coded configuration, with hooks / lifecycle events / enrichment generated from code
    - what exactly this will look like is still very fuzzy, but initial thinking is some properties of config could point to a TS module instead of being a static object / string. Or perhaps even all of them - we'll have to start sketching it out
- the "top" layer - the configuration format - will be used to generate the lower-level configuration. Configuration on the top will not consist of "whitelisted" low-level options, it will be translated. The idea here being that if we switch out configs lower down the stack, we shouldn't have to change the chap-level configs. We're treating top-level config schema as an interface which should very rarely change

Thoughts on open AI Agent Quality Engineer position

- Chaps has shifted focus from building our own agents to providing a platform for developers to create their own, becoming more of a "marketplace" model rather than an agent creator. I took a step back: does it still make sense to view "Quality" of our agents as the key differentiator between other platforms and us? In other words, are we still hiring for this specialized position?

  In my view, yes, but not for the original reasons we started this recruitment drive. The responsibility for this position has shifted from ensuring quality of our own bots to enabling external developers to deliver the best possible agents to our users.

  In my view, what makes or breaks a platform like Chaps - apart from adoption - is user (end user and dev alike) trust. Trust can be built by consistently delivering useful and safe agents to users. If we can guarantee that anything a user installs from our platform is observable / scored / meets a bar for quality, we can stand out from the rest. Quality should still be at the forefront of what we deliver to devs & users alike.

  My thoughts right now for "new" responsibilities are around owning chap quality & ranking signals (+ surfacing them), ensuring compliance & safety (so a "bad" or not-so-skilled developer cannot harm user trust), building out observability tooling for developers on the platform.

For example, Microsoft is already using "safety" as a selling point for their platform - "*one of the key advantages of using DeepSeek R1 or any other model on Azure AI Foundry is the speed at which developers can experiment, iterate, and integrate AI into their workflows," says Asha Sharma, Microsoft's corporate vice president of AI platform. "DeepSeek R1 has undergone rigorous red teaming and safety evaluations, including automated assessments of model behavior and extensive security reviews to mitigate potential risks*". This platform is slightly different of course, but still shows what is deemed important when hosting AI models & agents - albeit in an enterprise setting.

External agents give us "breadth", but inconsistent quality erodes trust. Enabling this ecosystem's quality should be the direction this position pushes us in.

(ref, ref)

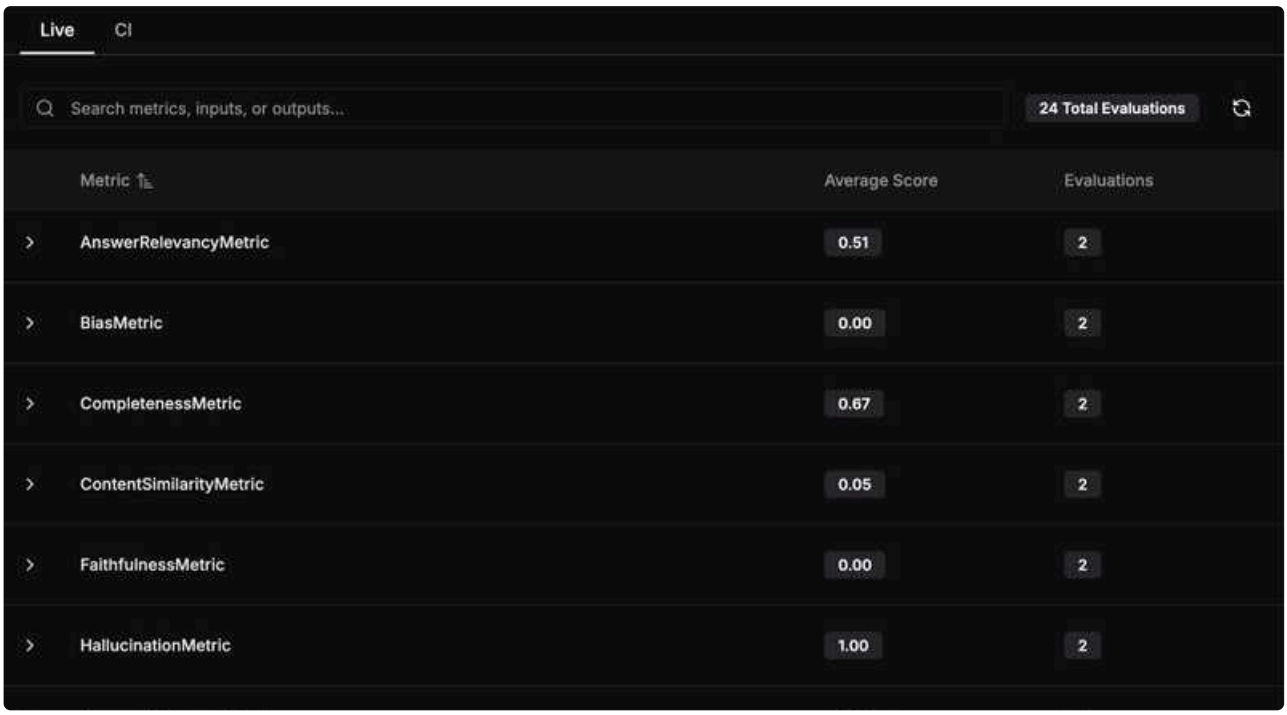
Mastra

We also talked about what would be important to understand about Mastra. I started the proto/exploration with the first question - can we capture all the steps (thinking, tool calls) that we capture now with Orchestra? A lot depends on the steps taken in Chaps.

Yes, we can. See code

I built a simple agent that has 2 tools - a geocode tool to take a city and return coordinates, and a tool to fetch aircraft in the air around certain coordinates. These both call external APIs and return structured data.

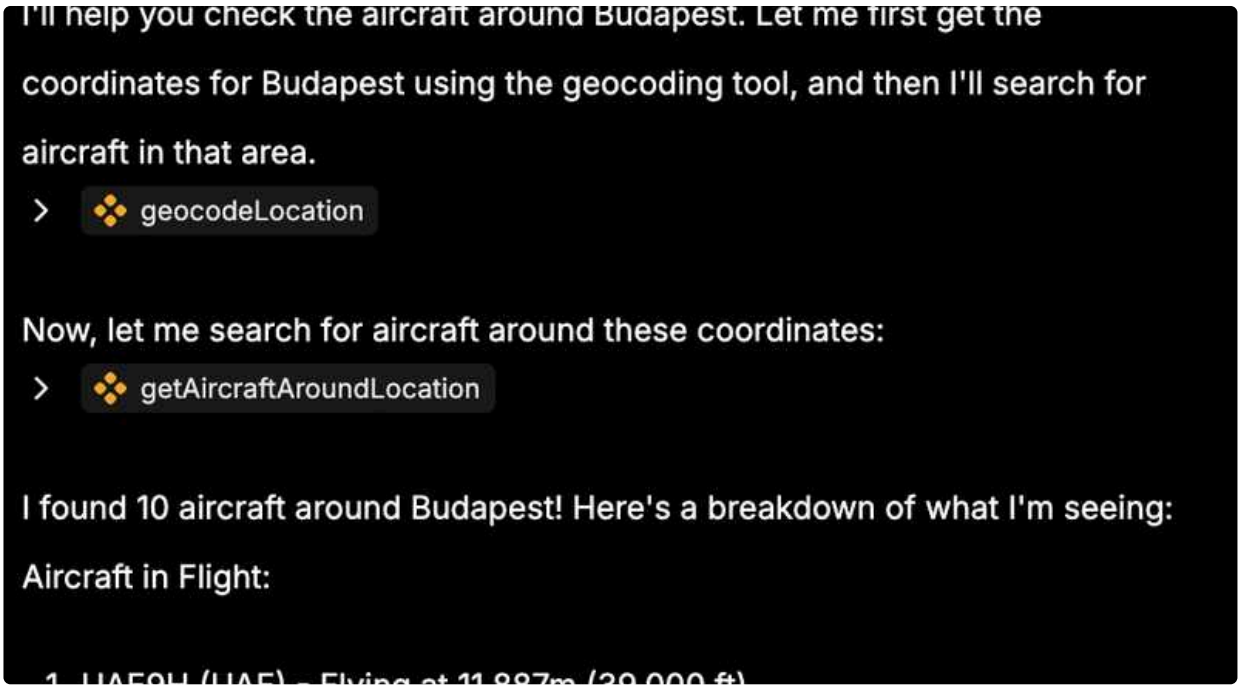
I added Mastra's eval metrics to see how they would perform and if they are useful. Mastra uses 2 types - calculated (simpler, string-based metrics) and LLM-judged. Evals can be seen on the playground as the agents run:



Taking a simple prompt

- Input

```
what aircraft are flying around Budapest right now?
```



- Output

```
I'll help you check the aircraft around Budapest. Let me first get the coordinates for Budapest using the geocoding tool, and then I'll search for aircraft in that area.

Now, let me search for aircraft around these coordinates:

I found 10 aircraft around Budapest! Here's a breakdown of what I'm seeing:

Aircraft in Flight:

    UAE9H (UAE) - Flying at 11,887m (39,000 ft)
    THY8HV (Turkey) - Flying at 8,694m (28,525 ft)
    UAE7V (UAE) - Flying at 11,582m (38,000 ft)
    BAW5 (UK) - Flying at 10,660m (34,973 ft)
    MSC2928 (Ukraine) - Flying at 10,973m (36,000 ft)
    RYR6RN (Ireland) - Just taking off/landing

Aircraft on Ground:

    WZZ84 (Hungary) - Wizz Air aircraft
    ISR781 (Israel)
    BEL4WF (Belgium)
    FIN1HA (Denmark)

Interesting observations:

    There's a good mix of both international and regional traffic
    Two UAE aircraft are passing through the area at similar altitudes
    Several aircraft are on the ground at what appears to be Budapest Ferenc Liszt International Airport
    Airlines represented include Wizz Air (WZZ), Emirates (UAE), Turkish Airlines (THY), British Airways (BAW), Ryanair (RYR), and others
    Most cruising aircraft are at typical commercial aviation altitudes between 28,000-39,000 feet

Would you like any specific information about any of these aircraft?
```

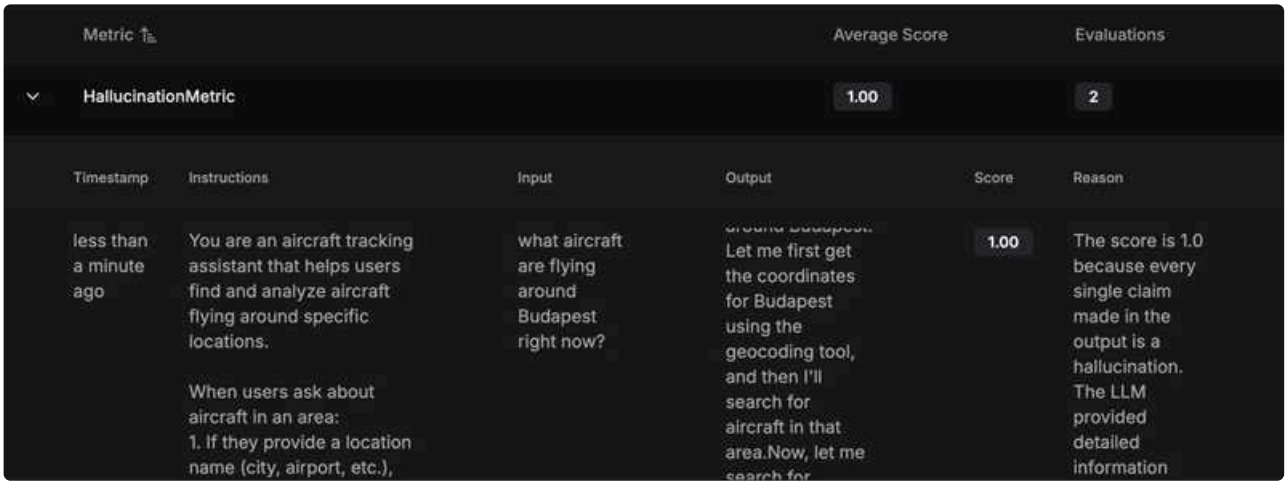
▼ Tool calls

```
1. First, let's get Budapest's coordinates:
  Tool calls:
    1. geocodeLocation:
      {
          "location": "Budapest"
      }
  Tool results:
    1.
      {
          "type": "tool-result",
          "toolCallId": "toolu_bdrk_01GbqrWfYZw5wTNa91c3gM2M",
          "toolName": "geocodeLocation",
          "args": {

```

```
            "location": "Budapest"
        },
        "result": {
            "latitude": 47.4813896,
            "longitude": 19.1460941,
            "display_name": "Budapest, Közép-Magyarország, Magyarország"
        }
    }

------

  1. getAircraftAroundLocation:
    {
        "latitude": 47.4813896,
        "longitude": 19.1460941,
        "radius_km": 50
    }
Tool results:
  1.
    {
        "type": "tool-result",
        "toolCallId": "toolu_vrtx_01XLYcCFFErhBu6tvD4Mxydc",
        "toolName": "getAircraftAroundLocation",
        "args": {
            "latitude": 47.4813896,
            "longitude": 19.1460941,
            "radius_km": 50
        },
        "result": {
            "time": 1751300454,
            "aircraft_count": 10,
            "aircraft": [
                {
                    "icao24": "471f35",
                    "callsign": "WZZ84",
                    "origin_country": "Hungary",
                    "time_position": 1751300454,
                    "last_contact": 1751300454,
                    "longitude": 19.2423,
                    "latitude": 47.86,
                    "baro_altitude": 3634.74,
                    "on_ground": false,
                    "velocity": 161.69,
                    "true_track": 32.53,
                    "vertical_rate": 13.98,
                    "sensors": null,
                    "geo_altitude": 3840.48,
                    "squawk": "4245",
                    "spi": false,
                    "position_source": 0
                },
```

```
                .....OMITTED.....

                {
                    "icao24": "4d222a",
                    "callsign": "RYR73NL",
                    "origin_country": "Malta",
                    "time_position": 1751300453,
                    "last_contact": 1751300453,
                    "longitude": 19.535,
                    "latitude": 47.8849,
                    "baro_altitude": 10363.2,
                    "on_ground": false,
                    "velocity": 200.78,
                    "true_track": 304.67,
                    "vertical_rate": 0,
                    "sensors": null,
                    "geo_altitude": 10835.64,
                    "squawk": "6171",
                    "spi": false,
                    "position_source": 0
                }
            ]
        }
    }
```

Some metrics did not make much sense, some were actually very useful. Take the "hallucination" metric for example:



Metric		Average Score	Evaluations
HallucinationMetric		1.00	2

Timestamp	Instructions	Input	Output	Score	Reason
less than a minute ago	You are an aircraft tracking assistant that helps users find and analyze aircraft flying around specific locations. When users ask about aircraft in an area: 1. If they provide a location name (city, airport, etc.),	what aircraft are flying around Budapest right now?	Let me first get the coordinates for Budapest using the geocoding tool, and then I'll search for aircraft in that area.Now, let me search for	1.00	The score is 1.0 because every single claim made in the output is a hallucination. The LLM provided detailed information

Reasoning provided for the hallucination score of 1.0 (meaning all of it is hallucination):

> The score is 1.0 because every single claim made in the output is a hallucination. The LLM provided detailed information about 10 specific aircraft around Budapest, including their flight codes, countries, altitudes, and statuses (in flight vs. on ground), but none of this information is supported by the provided context. The output also made additional unsupported claims about the mix of traffic, aircraft locations at Budapest Ferenc Liszt International Airport, and typical commercial aviation altitudes. Since there was no context provided that contained any information about aircraft near Budapest, all 16 verified statements were hallucinations, resulting in the maximum hallucination score of 1.0.

That isn't really helpful. However, digging into how this works, *if the eval is defined on the agent itself*, it should get context populated during runs, and this metric is judged based on the context. The context here would be outputs of tool calls. For some reason it wasn't added, therefore the judge correctly stated it is all a hallucination. I'll look into why it isn't being populated - the UI and console logs show tool calls are being made, and structured results returned, which are used in the final answer.

Evals can be defined elsewhere to be run, for example, as part of a test suite. Defining them on the agent will result in them being run every time the agent is run - which is great for dev, not so much for prod.

A metric that made much more sense is `PromptAlignment`, getting almost perfect scores, with the reasoning

> The score is 1 because the output perfectly follows all applicable instructions. It properly uses geocoding to find Budapest's location, clearly reports the total number of aircraft found (10), consistently provides altitude measurements in both meters and feet for aircraft in flight, and neatly organizes the information by separating aircraft in flight from those on the ground. The response is comprehensive, well-structured, and adheres to all the specified formatting requirements.

`AnswerRelevancyMetric`:

> The score is 0.46 because the response contains a mix of directly relevant information (6 aircraft currently flying around Budapest with details) and partially relevant information (4 aircraft on the ground, which aren't technically "flying" but are in the area). The response directly answers the core question by identifying specific aircraft in flight around Budapest with their altitudes and airlines, but also includes additional context about grounded aircraft and general observations that aren't strictly about "flying" aircraft. The follow-up question at the end is completely irrelevant to answering the original query, further reducing the overall relevance score.

The LLM judge seems pretty strict

This is a simple example, it uses `claude3.7-sonnet`, I would have been surprised if it didn't get high scores.

Mastra also has built-in tracing, which was not working reliably with Openrouter. With OpenAI we could see traces. If we can get this working, local development could be done almost completely in the playground, running as if it were on the chaplet.

Our current setup with the communicator agent translating inputs of a git diff can be modeled nicely with the control flow - https://mastra.ai/en/docs/workflows/control-flow - user input to agent, after that step to get the diff, output of that goes to communicator, output then goes to user. Each step can be either a method, a tool, or an agent.

MCP: if MCP servers are defined on an agent (via MCPClient), starting them is handled by Mastra. This makes things simpler for us by not having to worry about manually starting them.

We can very easily (= built in to the framework) expose tools, agents, and workflows in an MCP server. To scale out Chaps, we would want to move from the 1 MCP server / chaplet model to the 1 MCP server in Chaps model. Similarly, for tools that don't depend on the local chaplet state, we could expose common chaps tooling via our own internal MCP server(s). By reducing services on the chaplet we can reduce complexity (install, update chaps), with the caveat that versioning becomes much more important. It might even make sense to move all current MCP servers off chaplets and store state only, possibly by just proxying MCP requests via our services, and adding auth etc. on the proxy layer, not on the chaplet layer → passing `RuntimeContext` which is set via Agent SDK.

Next steps

- Sipi and I talked about what would be the best current chap to test with Mastra, and agreed it should be the Linear chap. I'll work on this tomorrow
- A lot of the enrichment we do today with custom code is available built-in to the framework. I intend to use Mastra's methods for the Linear chap proto (above)
- MCP - client and server. Mastra supports agents being an MCP server themselves, and also has built-in support for calling MCP tools. An interesting angle of this would be if a chap could be an MCP server and "cross-chap" calling would just be adding another chap as an MCP to the chap we're chatting with